Government Engineering College,Ramanagara

Department of Electronics & Communication Engineering

INTERNET OF THINGS LABORATORYMANUAL

[BEC657C]

(VI SEM)

PreparedBy:

INCHARA K

AssistantProf, Dept of ECE

Government Engineering College, Ramanagara

Department of Electronics & Communication Engineering

For the Year: 2025-2026

Department of Collegiate and Technical Education

GOVERNMENT ENGINEERING COLLEGE, RAMANAGARA

Vision

"To become a premier Engineering college by providing quality technical education and research with professional ethics embedded with human values."

Mission

- To constantly improve quality of education system that produce engineers having good technical capabilities with ethical and human values.
- To encourage faculty and students to engage in significant research and consultancy by nurturing a good eco-system.
- To strengthen industry institute collaboration for promoting teamwork, internship, training and entrepreneurship.
- To enhance educational opportunities for the students from ruraland weaker sections of the society.

ELECTRONICS & COMMUNICATION ENGINEERING DEPARMENT

Vision

"To become a pioneer in technical education and research by producing global professionals with ethics and human values for societal needs".

Mission

- To impart excellence in technical education and research by providing innovation solutions to societal needs.
- Imparting Center of Excellence in State-Of-Art Infrastructure to make competent graduates by providing professionally committed Faculties.
- Inculcate Ethical values, Technical Capabilities & Leadership abilities for meeting the current and future demands of Industry and Society.

NBA Programme Outcome

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Program Specific Outcomes (PSOs)

PSO1: Apply knowledge and skills to design and develop innovative embedded systems and IoT solutions, addressing real-world challenges and practical applications.

PSO2: Integrate and implement hardware and software techniques to analyze, optimize, and create solutions for signal processing, advanced communication, and VLSI system development.

Course syllabus

Sl. No.	Experiments
1.	i) To interface LED/Buzzer with Arduino /Raspberry Pi and write a program to 'turn ON' LED for 1 sec after every 2 seconds. ii) To interface the Push button/Digital sensor (IR/LDR) with Arduino /Raspberry Pi and write a program to 'turn ON' LED when a push button is pressed or at sensor detection.
2.	i) To interface the DHT11 sensor with Arduino /Raspberry Pi and write a program to print temperature and humidity readings. ii) To interface OLED with Arduino /Raspberry Pi and write a program to print its temperature and humidity readings.
3.	To interface the motor using a relay with Arduino /Raspberry Pi and write a program to 'turn ON' the motor when a push button is pressed.
4.	i) Write an Arduino/Raspberry Pi program to interface the Soil Moisture Sensor. ii) Write an Arduino/Raspberry Pi program to interface the LDR/Photo Sensor.
5.	Write a program to interface an Ultrasonic Sensor with Arduino /Raspberry Pi.
6.	Write a program on Arduino/Raspberry Pi to upload temperature and humidity data to thingspeak cloud.
7.	Write a program on Arduino/Raspberry Pi to retrieve temperature and humidity data from thingspeak cloud.
8.	Write a program to interface LED using Telegram App.
9.	Write a program on Arduino/Raspberry Pi to publish temperature data to the MQTT broker.
10.	Write a program to create a UDP server on Arduino/Raspberry Pi and respond with humidity data to the UDP client when requested.
11.	Write a program to create a TCP server on Arduino /Raspberry Pi and respond with humidity data to the TCP client when requested.
12.	Write a program on Arduino / Raspberry Pi to subscribe to the MQTT broker for temperature data and print it.

Internet of Things

One of the greatest inventions of human civilization of all time is the global (worldwide) computer network - the Internet. Like most other inventions, the Internet has its good and bad sides. When it comes to the good sides of the Internet, it first and foremost facilitates all forms of communication: Person to Person (P2P); human with the computer and the computer with the computer. With the development of the Internet, new types of communication are emerging: human communication with "smart things", which, together with human communication with a computer, represents Person to Machine communication (P2M); communication between "smart things", which together with computer-to-computer communication and computer communication with "smart things," make up a new category of communication Machine to Machine (M2M). Ordinary objects (things or devices) used to need to be manually activated or programmed. This is no longer the case today, as most of these objects come automated, with pre-built "smart" components that can make their own decisions (without human presence). Internet of Things (IoT) or "Smart Things" are machines, robots, sensors, controls, etc. s.

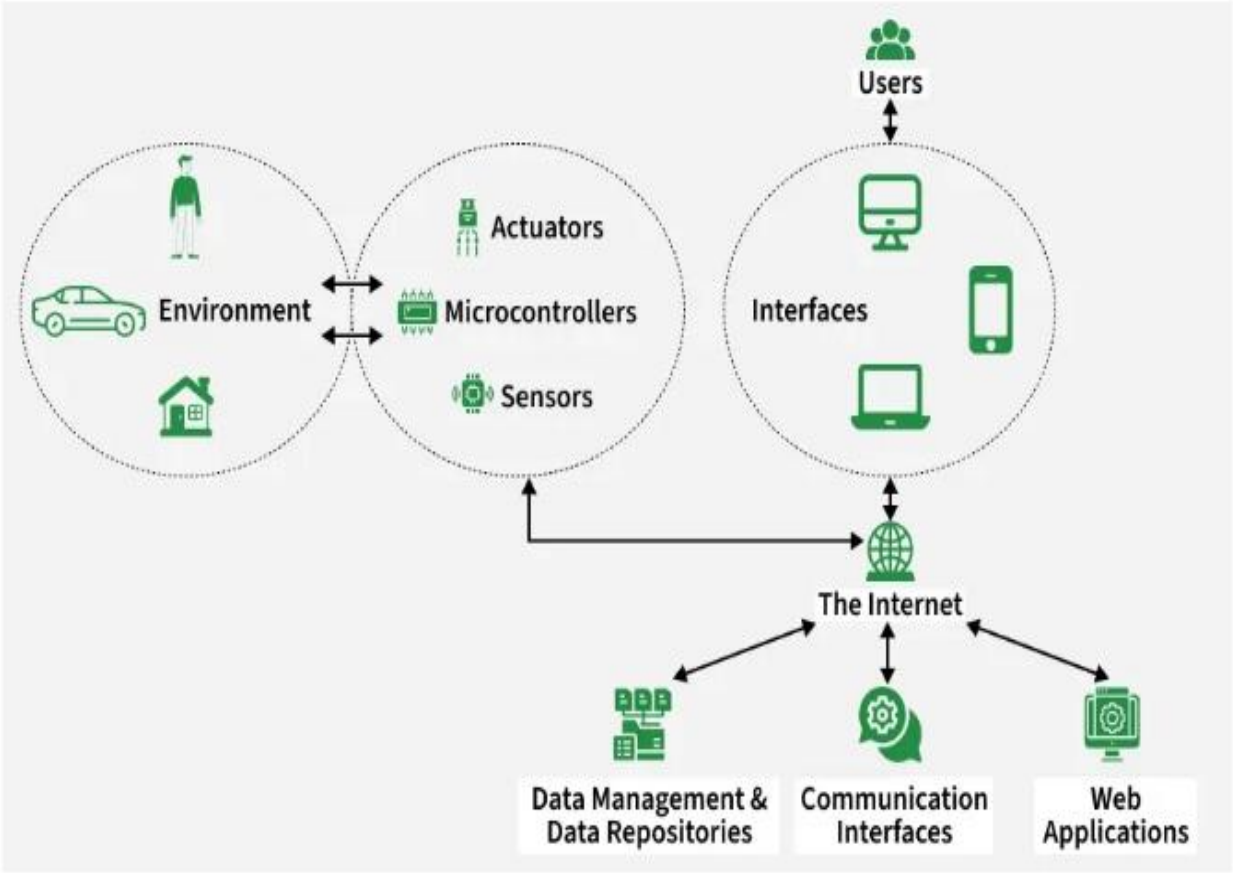
The Internet of Things (IoT) is a network of physical objects embedded with sensors, software, and communication technologies, enabling them to connect to the Internet and exchange data automatically without direct human involvement.

- Focuses on connecting everyday physical objects such as devices, machines, and systems to computer networks
- Enables real-time data collection, monitoring, and automated actions
- Plays a key role in modern computer networks by extending Internet connectivity beyond traditional computers
- Improves efficiency, accuracy, and convenience in daily life and industrial applications

Sensors collect data and send it via the network to edge or cloud systems where the data is processed and analyzed. Results are delivered to users through apps or used to automatically drive actuators and systems.

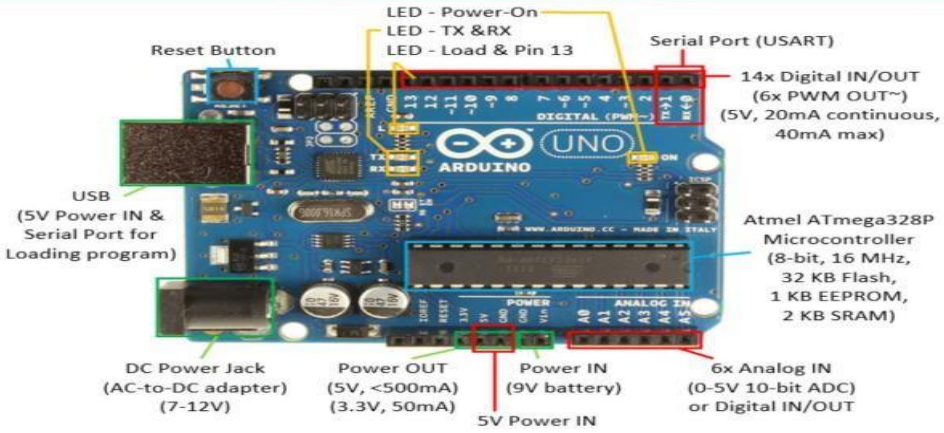
IoT applications span across domains of industrial control, retail, energy, agriculture, etc. This

module provides the theoretical and practical aspects of interfacing sensors and actuators, making informed world of Things speaking to each other.



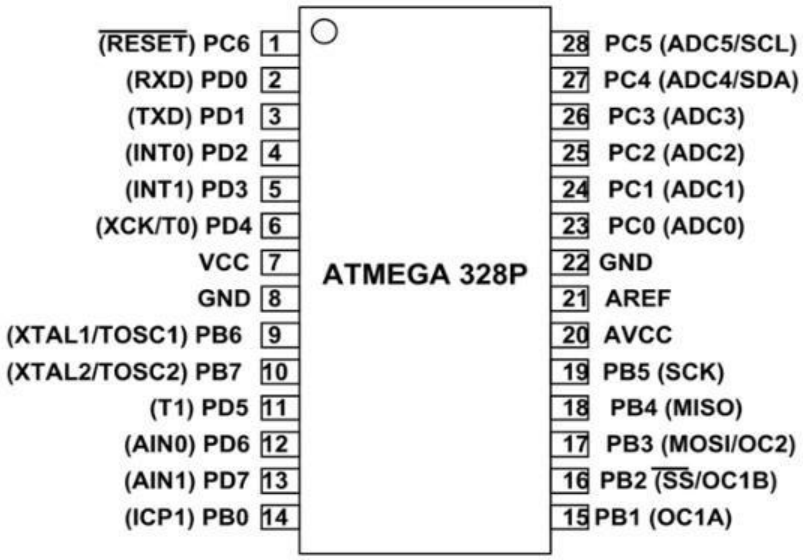
Introduction to Arduino board

"Arduino is open-source electronics prototyping platform based on flexible,easy-to-use hardware and software. It is intended for artists, designers, hobbyists, and anyone interested in creating interactive objects or environments." The mother site for Arduino is http://arduino.cc.



An Arduino UNO board contains the following:

An Atmel(bought over by Microchip in 2016)ATmega328P8-bitMicrocontroller with 32KB of Flash memory for programs, 2KB of SRAM for data, and 1KB of EEPROM for non-volatile data. The clock speed is 16MHz.



- PowerIN: The power to Arduino board can besapplied via:

◆ USB Connector (5V): also double as serial port to communicate with the computer (e.g., for loading program).

◆ DC Power Jack (7-12ViaAC-to-DCadapterorbatterypack).

◆ Vin/GND pins (7-12V with9V battery).

◆ Power by 5V supply: Although the 5V pin is meant for powering external components, you could power the Arduino via the 5V/GND pins. This is not recommended because you could get the polarity and voltage wrong and damage your arduino. A better way is to connect your 5V power supply to the USB connector. Get an old USB cable, cut the plug off the other end, and connect Redwireto5Vsupply and Black wire to GND. USB connect or is better than the crappy header pins and you can't mess up the polarity.

- Power OUT (5V and 3.3V pins): The board regulated 5V and 3.3V output for powering external components.

◆ If the board is powered via USB, the total current available for "on-board" and "external use via the 5V power out pin" is 500mA (and much less for the other power sources).

◆ The current available at 3.3V pin is 50mA.

- On-board Built-in LEDs: These LEDs are useful in debugging.

◆ Power-on LED: indicate that the Arduino board is receiving power.

◆ Load cumPin-13LED: Flicker during sketch upload. Also connected to Digital Pin 13

for program testing and debugging.

◆ TX and RX LEDs: indicate communication between the Arduino board and your computer.

◆ Flicker during sketch upload and serial communication.

- Reset Button: to reset/restart the program.
- 14xDigitalINPUT/OUTPUT pins(numbered from 0to13):

◆ Use function pinMode(0-13, INPUT|OUTPUT) to configure the pin for input or output; and digitalRead(0-13) or digitalWrite(0-13,HIGH|LOW) to read or write.

◆ Each pin operates at 5V at HIGH and 0V at LOW. It can provide or receive a maximum current of 40mA, but only 20mA continuous, which is merely sufficient to drive a single-color LED at 20mA for full brightness continuously.

◆ Total current for the chipset shall not exceed 200mA, i.e., driving 10 single-color LEDs at 20mA.

◆ Pin 13: There is a built-in LED connected to Pin 13, under Pin 13.It is useful for debugging.

◆ PWM Output: 6 of the pins(pins 3,5,6,9,10and11, marked with~) can produce PWM (Pulse Width Modulated) output (i.e., square wave)

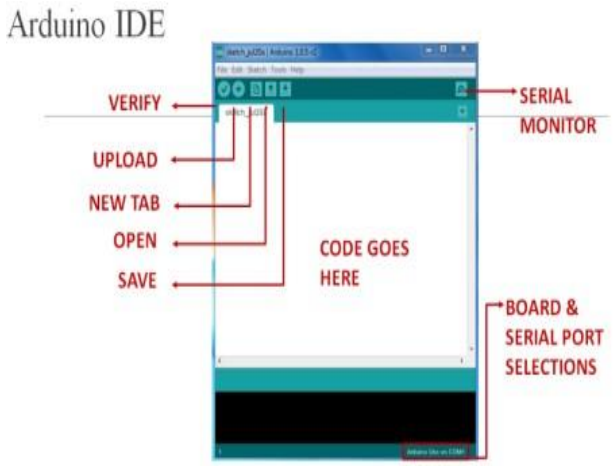
- 6x Analog INPUT pins (A0 to A5): By default, each pin can measure between 0V to 5V with 10-bit of resolution via a analog-to-digital converter (ADC). The upper range can be changed using the AREF pin and analog Reference() function. These analog pins are labeled A0 to A5 (whereas digital pins are labeled 0 to 13). We can use pinMode(A0-A5, INPUT) (optional but nice to have) and analog Read(A0-A5) to read the 10-bit analog input [0,1023].TheprimarypurposeoftheanaloginputpinsA0-A5istoreadanalog input. However, these pins can also function exactly like digital pins 0-13 for digital input/output. We can use pinMode(A0-A5,INPUT/OUTPUT) to set its I/O mode,andanddigitalRead(A0-A5)ordigitalWrite(A0-A5,HIGH|LOW)toreador write.
- 1xProgrammable Serial Port (USART):UseDigitalPin0forRXandDigital Pin 1 for TX; or the USB connection (for communicating with the PC such as loading program).
- Interrupts: Pins2 and 3 can be used for "external" interrupt and mapped to INT0 and

INT1. All 20 pins (14x digital + 6x analog) can be used for "pin-change" interrupt. Brief summary of digital/analog pins and read/write functions:

- Pin Mode (0-13|A0-A5, INPUT|OUTPUT): Set the pin to INPUT or OUTPUT. Digital pins are numbered from 0 to 13.Analog pins are numbered A0 to A5.
- Digital Read (0-13|A0-A5), digital Write (0-13|A0-A5, HIGH|LOW): applicable to all digital and analog pins.
- Analog Write (3|5|6|9|10|11, duty Cycle): applicable to PWM digital pins only, where duty Cycle= [0,255], 0 for low and 255 for high.
- Analog Read (A0-A5): read 10-bit input in the range of [0, 1023] for 0-5V from analog input pins. Protecting the Arduino Board:

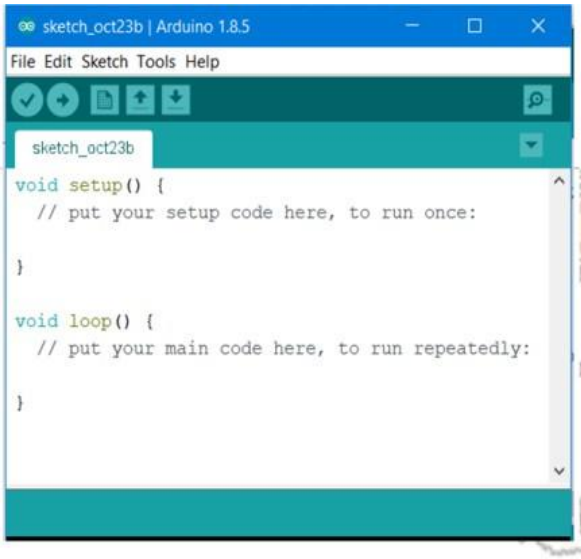
◆ Many of students throw the Arduino board around like a soft toy, and short the connections underneath. Buy a plaster casting or simply use masking tape to cover the base of the Arduino board.

◆ Disconnect the power/USB cable to the Arduino board, when you are connecting the circuitry. This also protect you precious PC!



Basic Coding Structure

- *setup()* function
  - Called when a sketch starts.
  - To initialize variables, pin modes, start using libraries, etc.
  - Will only run once, after each power-up or reset of the Arduino board.
- *loop()* function
  - Loops consecutively.
  - Code in the loop() section of the sketch is used to actively control the Arduino board.
- *Commenting*
  - Any line that starts with two slashes (//) will not be read by the compiler, so you can write anything you want after it.



- ***pinMode()***

  Instruction used to set the mode (INPUT or OUTPUT) in which we are going to use a pin.

  Eg: pinMode (13, OUTPUT);

  ie. setting pin13 as output.

- ***digitalWrite()***
  - Write a HIGH or a LOW value to a digital pin.
  - Eg: digitalWrite (11, HIGH);
  - ie. setting pin 11 to high.

- ***digitalRead()***
  - Reads the value from a specified digital pin, either HIGH or LOW
  - Eg: int inPin=7;
    val = digitalRead(inPin);
  - ie. reads the value from inPin and assigns it to val.

- ***delay()***
  - Pauses the program for the amount of time (in milliseconds) specified as parameter.
  - Eg: delay(1000);
  - ie. waits for a second (1000 ms = 1 s)

Introduction to NodeMCU

The NodeMCU(Node MicroControllerUnit) is an open-sources of the ware and hardware development environment built around an inexpensive System-on-a-Chip (SoC) called the ESP8266. The ESP8266, designed and manufactured by Espressif Systems, contains the crucial elements of a computer: CPU, RAM, networking(WiFi),and even a modern operating system and SDK. That makes it an excellent choice for Internet of Things (IoT) projects of all kinds.

Similar to NodeMCU, the Arduino hardware is a microcontroller board with a USB connector, LED lights, and standard data pins. It also defines standard interfaces to interact with sensors or other boards. But unlike NodeMCU, the Arduino board can have different types of CPU chips (typically an ARM or Intel x86 chip) with memory chips, and a variety of programming environments. There is an Arduino reference design for the ESP8266 chip as well. However, the flexibility of Arduino also means significant variations across different vendors. For example, most Arduino boards do not have WiFi capabilities,and some even have a serial data port

instead of a USB port.

- Power Pins There are four power pins. VIN pin and three 3.3V pins.

◆ VIN can be used to directly supply the NodeMCU/ESP8266 and its peripherals. Power delivered on VIN is regulated through the onboard regulator on the NodeMCU module – you can also supply 5V regulated to the VIN pin

◆ 3.3V pins are the output of the onboard voltage regulator and can be used to supply power to external components.

- GND are the ground pins of NodeMCU/ESP8266

- I2C Pins are used to connect I2C sensors and peripherals. Both I2C Master and I2C Slave are supported. I2C interface functionality can be realized programmatically, and the clock frequency is 100kHz at a maximum. It should be noted that I2C clock frequency should be higher than the slowest clock frequency of the slave device.

- GPIO Pins NodeMCU/ESP8266has17GPIOpinswhichcanbeassignedto functions such as I2C, I2S, UART, PWM, IR Remote Control, LED Light and Button programmatically. Each digital enabled GPIO can be configured to internal pull-up or pull-down, or set to high impedance. When configured as an input, it can also be set to edge-trigger or level-trigger to generate CPU interrupts.

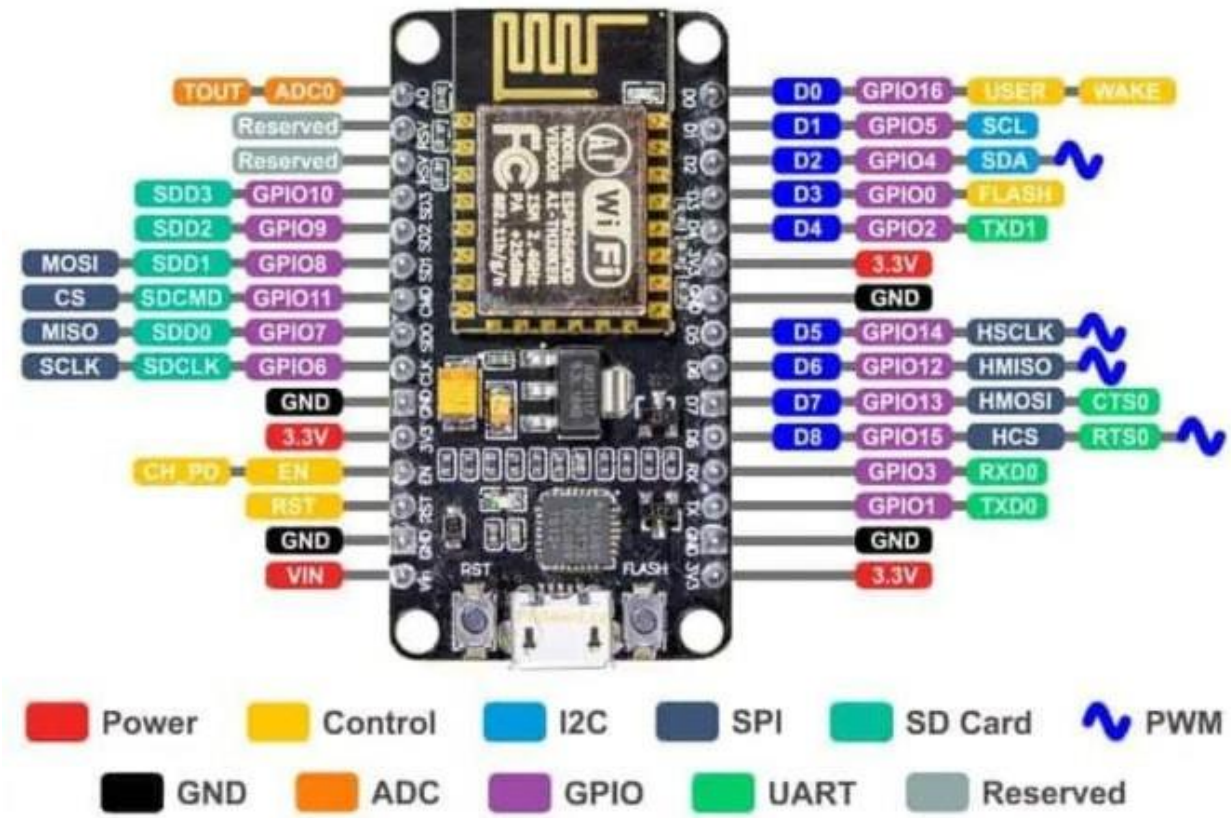
NodeMCU Pin out and Functions

- ADC Channel The NodeMCU is embedded with a 10-bit precision SARADC. The two functions can be implemented using ADC. Testing power supply voltage of VDD3P3 pin and testing input voltage of TOUT pin. However, they cannot be implemented at the same time.

- UART Pins NodeMCU/ESP8266has2UART interfaces(UART0andUART1) which provide asynchronous communication (RS232 and RS485), and can communicate at up to 4.5 Mbps. UART0 (TXD0, RXD0, RST0 & CTS0 pins) can be used for communication. However, UART1 (TXD1 pin) features only data transmit signal so, it is usually used for printing log.

- SPI Pins NodeMCU/ESP8266 features two SPIs (SPI and HSPI) in slave and master modes. These SPIs also support the following general-purpose SPI features:

- 4 timing modes of the SPI format transfer
- Upto80 MHz and the divided clocks of 80 MHz.



- Upto 64-ByteFIFO

- SDIO Pins NodeMCU/ESP8266 features Secure Digital Input/Output Interface(SDIO)whichisusedtodirectlyinterfaceSDcards.4-bit25MHzSDIO v1.1 and 4-bit 50 MHz SDIO v2.0 are supported.
- PWM Pins The board has 4 channels of Pulse Width Modulation (PWM). The PWM output can be implemented programmatically and used for driving digital motors and LEDs. PWM frequency range is adjustable from 1000 µs to 10000 µs (100 Hz and 1 kHz).
- Control Pins are used to control the NodeMCU/ESP8266. These pins include Chip Enable pin (EN), Reset pin (RST) and WAKE pin.

- EN: The ESP8266 chip is enabled when EN pin is pulled HIGH. When pulled LOW the chip works at minimum power.

- ◆ RST: RST pin is used to reset the ESP8266 chip.
- ◆ WAKE: Wake pin is used to wake the chip from deep-sleep.

- Control Pins are used to control the NodeMCU/ESP8266.Thesepins include Chip Enable pin (EN), Reset pin (RST) and WAKE pin.

- ◆ EN: The ESP8266 chip is enabled when EN pin is pulled HIGH. When pulled LOW the chip works at minimum power.
- ◆ RST: RST pin is used to reset the ESP8266 chip.
- ◆ WAKE: Wake pin is used to wake the chip from deep-sleep.

USB to Serial Converter–CP2102 or CH340G

In corporated into each NodeMCU is a USB to Serial Converter. The official design is based on the CP2102 chip set and offers the best compatibility. Genuine boards use the CP2102 chip set including the officially licensed Amica NodeMCU modules. The other common USB to Serial Converter used is the CH340G which is common on the lower-priced modules including the LoLinunits. Other designs may use drivers including the FTDI chipset, but those designs are rare.

Depending on the Operating System you are using with the NodeMCU,the appropriate driver must be installed. Generally, Windows 10 immediately recognizes the CP2102 chipset while the CH340G may require separate installation.

- Drivers for the CP2102 are available for download from the Silicon Labs support site. Drivers constantly evolve and ensuring and installing the most recent version in your development environment minimum issues. Drivers are available for Windows, Mac, Linux, and Android. We also have a local copy of the CP2102 drivers(v10.1.8) available locally for download. You are always best to visit the original manufacturer to ensure you are receiving the most recent versions of the driver.

Softwares required for the IOT lab

- Ch-340driver for Arduino
- CP210x_VCP_Windows
- XAMPP Latest Version

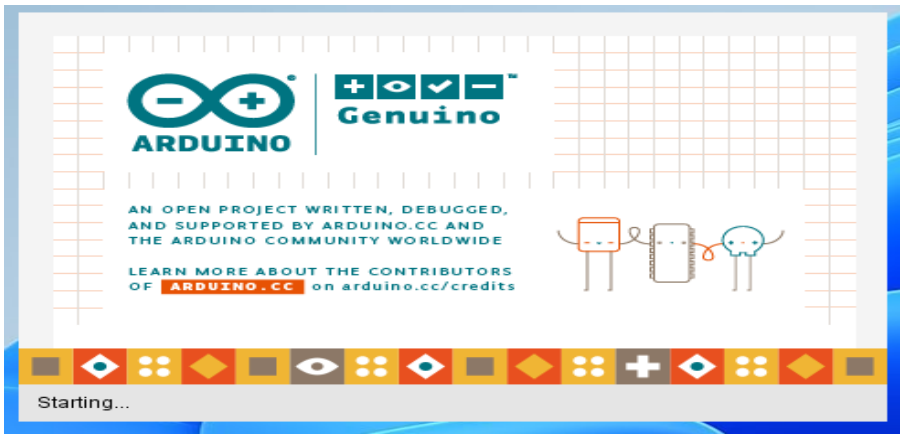
- arduino-1.8.7

Libraries required for the IOT lab

- Adafruit_BusIO
- Adafruit_GFX_Library
- Adafruit_SSD1306
- adidax-dht11-b7fbbcd
- Arduino-LiquidCrystal-I2C-library-master
- DHT
- LiquidCrystal_I2C
- pubsubclient-master
- thingspeak-arduino-master

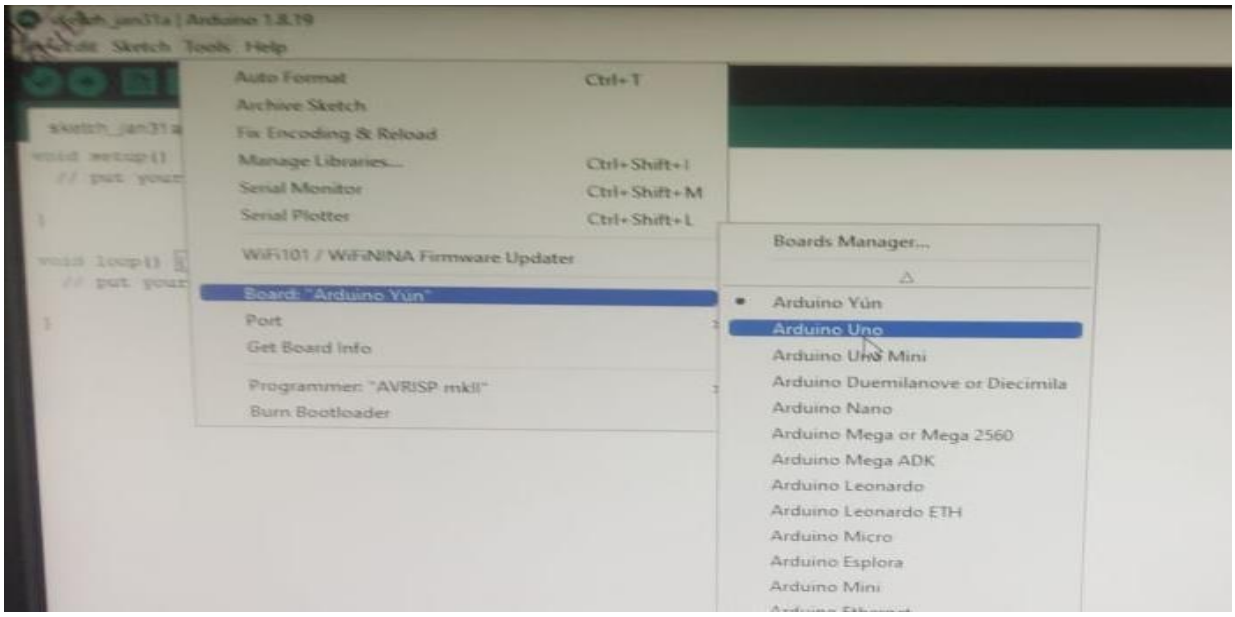
Procedure to Create Project in Arduino

1. Open The Arduino Software.

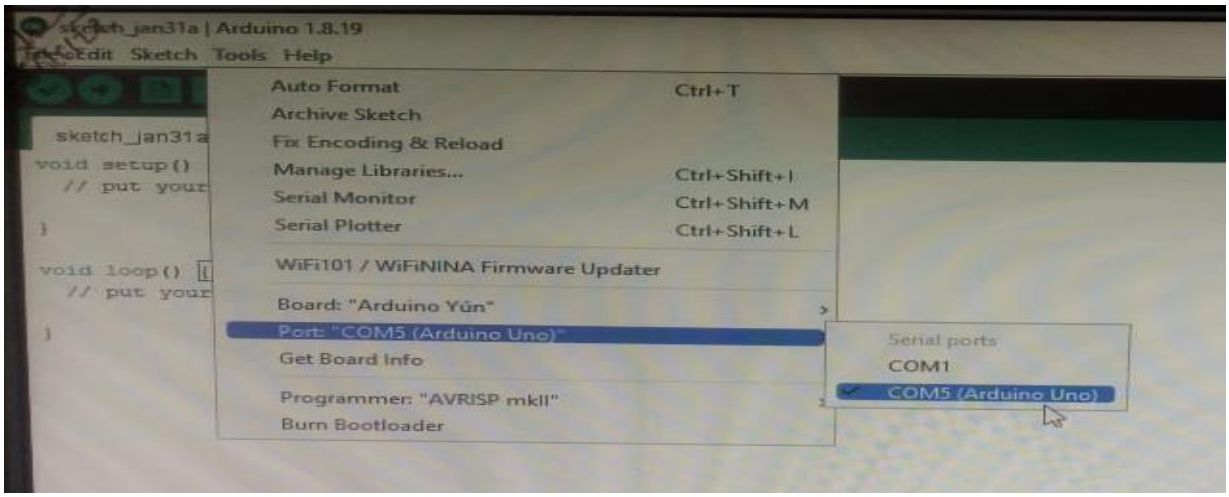


2. Open the new file.

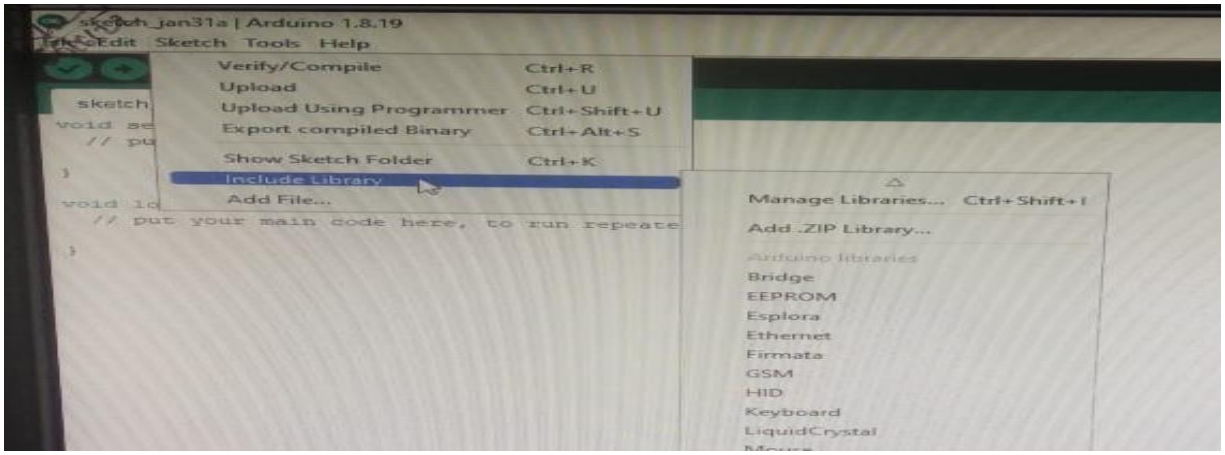
3. Select the board as arduino uno in software tools.



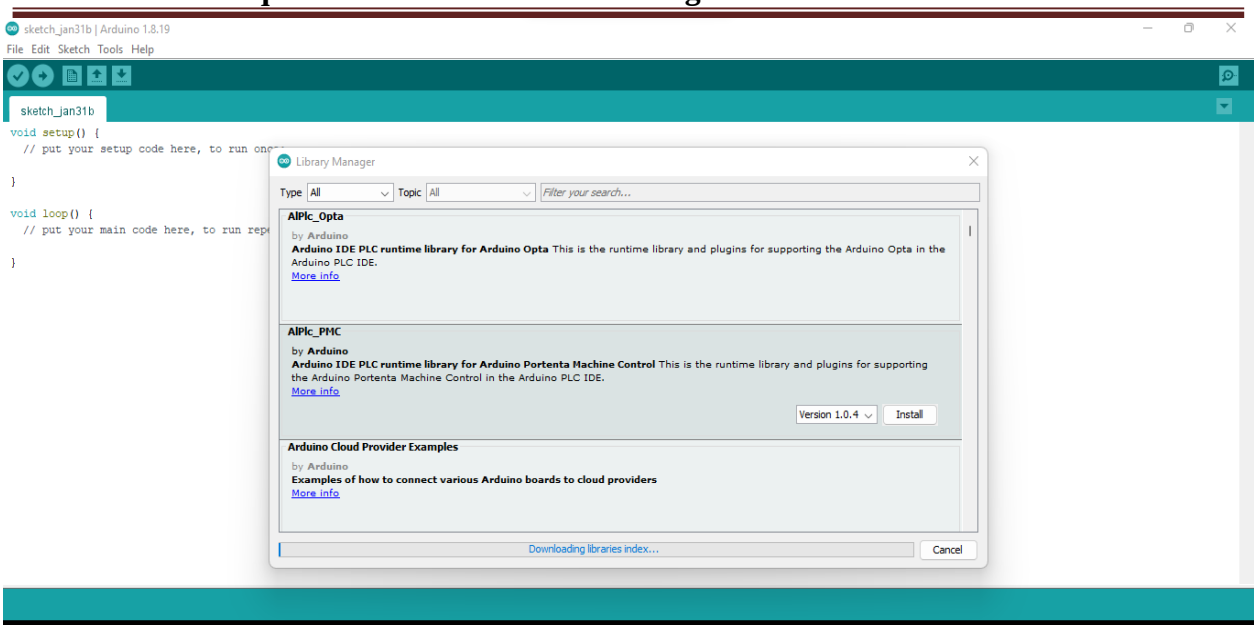
4. Select the exact comport connected to your computer system from tool menu.



5. Include the required library from the sketch menu.

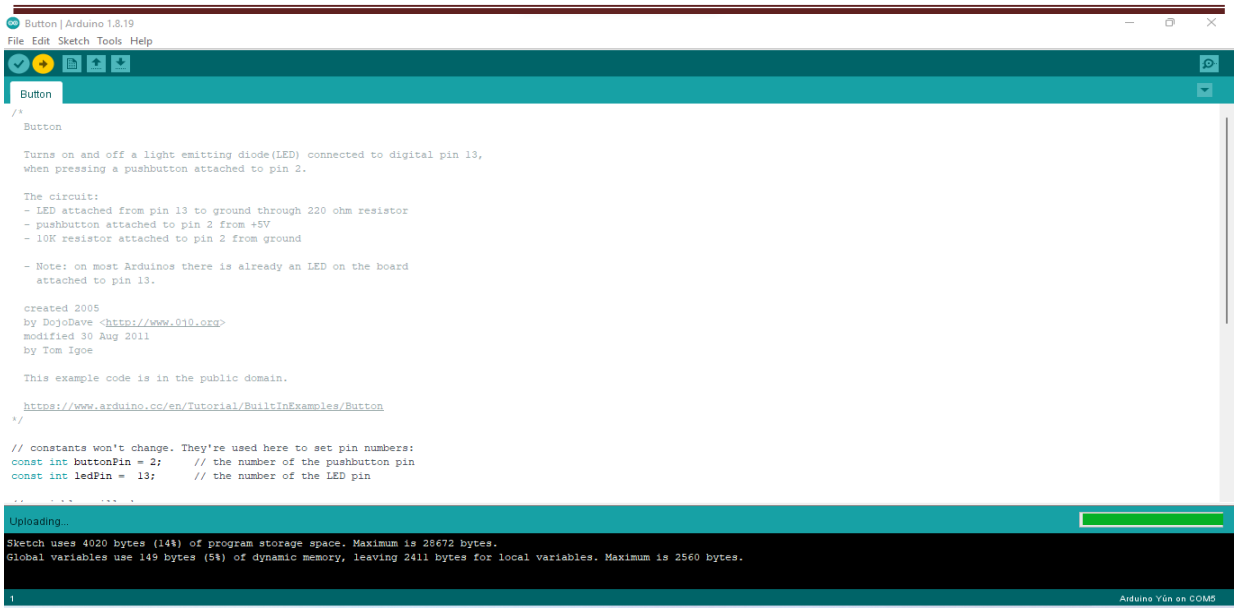


6. Add the required libraries from manage libraries.

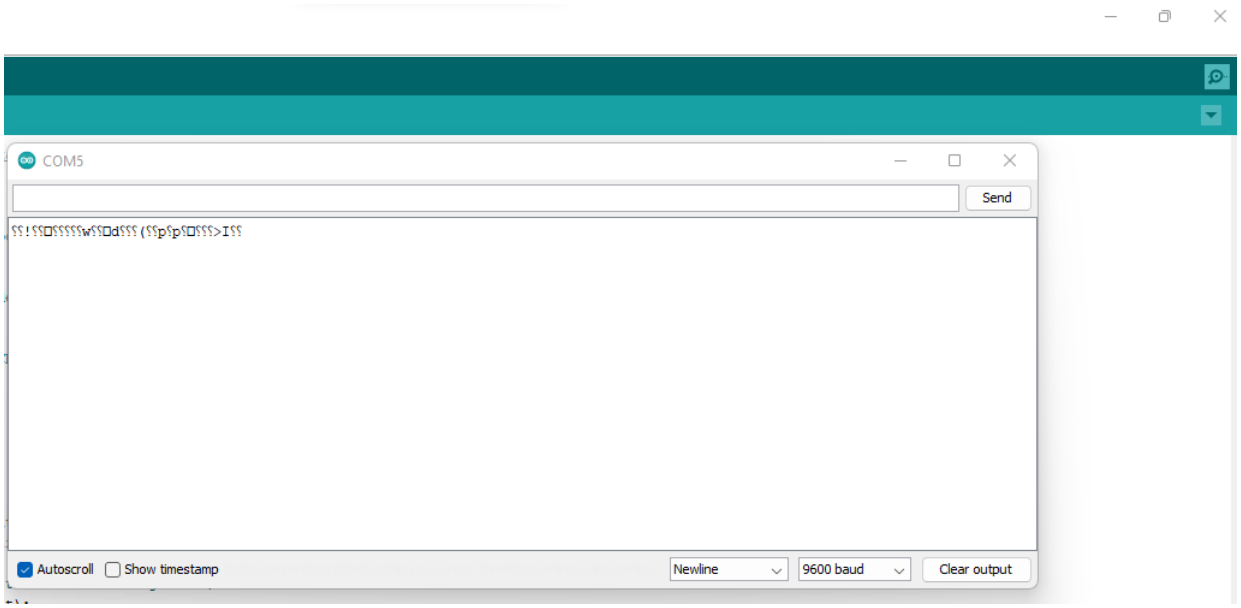


7. Verify the programme by clicking the tick button.

8. Compile the programme clicking the right arrow button.



9. Observe the Output On Serial Monitor Or On The Circuit As Per Your Program.



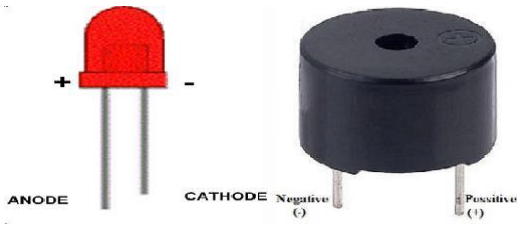
EXPERIMENT – 1

I. To interface LED/Buzzer with Arduino/Raspberry Pi and write a program to turn ON LED for 1 sec after every 2 seconds.

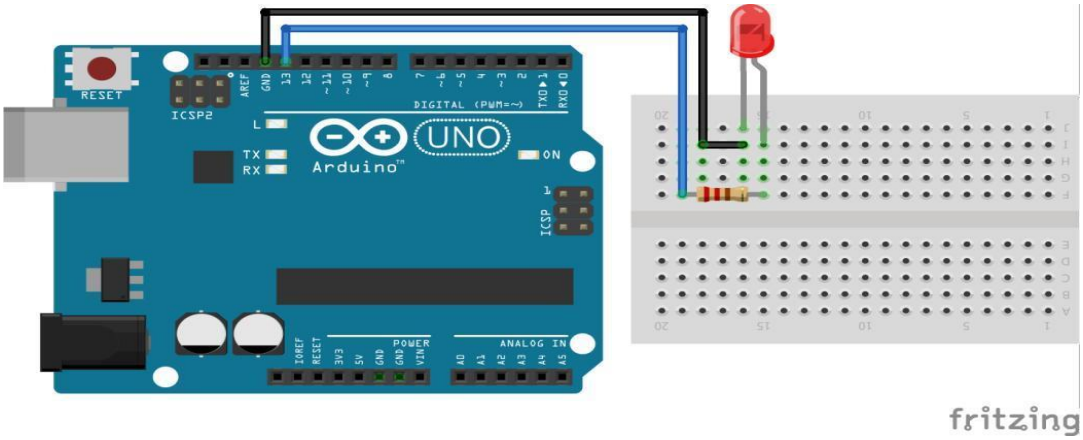
AIM: To write a program to turn ON LED/ Buzzer for 1 sec after every seconds with Arduino/ Raspberry Pi.

COMPONENTS REQUIRED:

1. Arduino UNO/ Raspberry Pi.
2. LED/ Buzzer.
3. Resistor (220Ω)
4. Connecting cable or USB cable.
5. Breadboard.
6. Jumper wires.



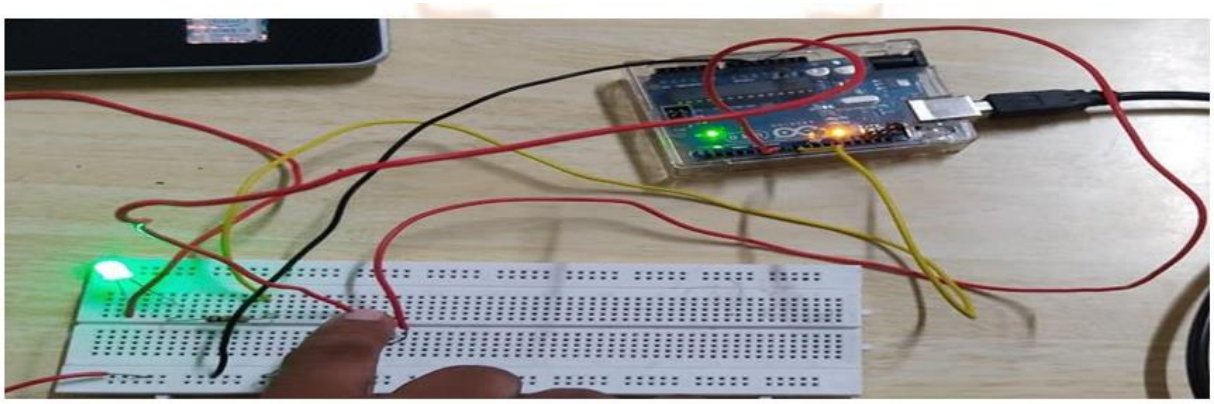
(a) LED (b) Buzzer



CODING:

```
int LED =8;
void setup()
{
pinMode(LED,OUTPUT);
}
void loop()
{
digitalWrite(LED,HIGH);
delay(1000);
digitalWrite(LED,LOW);
delay(1000);
}
```

RESULT: LED/Buzzer is successfully controlled by Arduino UNO/ Raspberry Pi Board.



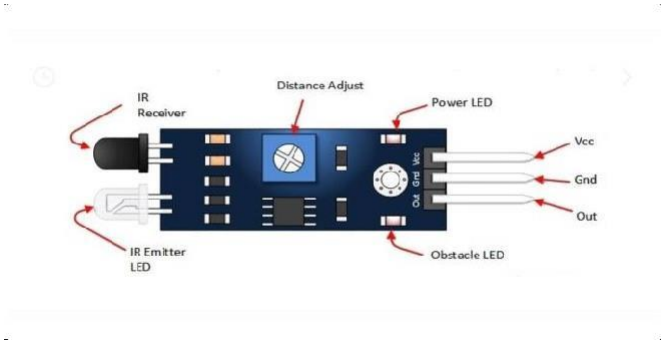
II.To interface Push button/Digital sensor (IR/LDR) with Arduino UNO and write a program to turn ON LED when push button is pressed or at sensor detection.

AIM: To write a program to turn ON LED at sensor (IR) detection using Arduino UNO.

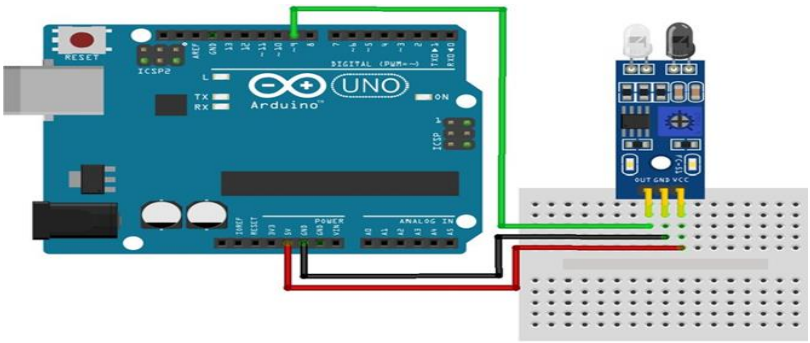
COMPONENTS REQUIRED:

1. Arduino UNO.
2. IR.
3. LED.
4. Resistor (220Ω)
5. Connecting cable or USB cable.
6. Breadboard.
7. Jumper wires.

CONNECTION DIAGRAM:



IR sensor



CODING:

Embedded C

```
void setup()
{
Serial.begin(9600);
pinMode(2, INPUT);
Arduino_h pinMode(10, OUTPUT);
}

void loop()
{
int sensor_state =digitalRead(2);
 if (sensor_state ==1)
{
Serial.println("Motion detected");
digitalWrite(10, HIGH);
}
else
{
Serial.println("Motion detected");
digitalWrite(10, LOW);
}
}
```

RESULT: Object detection by interfacing IR with Arduino UNO has successfully executed

EXPERIMENT – 2

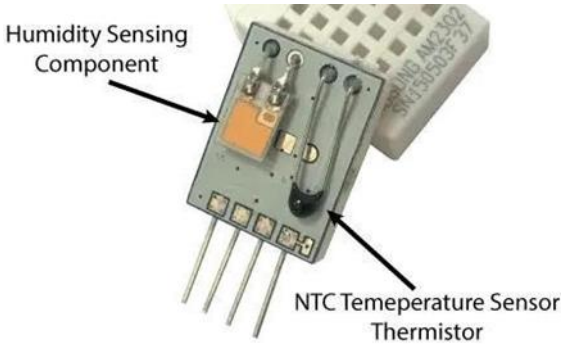
I.To interface DHT11 sensor with Arduino/Raspberry Pi and write a program to print temperature and humidity readings.

AIM: Write a program to interface DHT11 with Arduino/Raspberry Pi to print temperature and humidity readings.

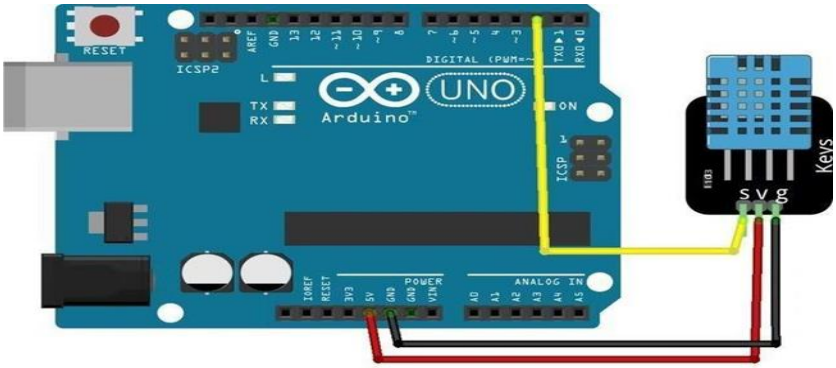
COMPONENTS REQUIRED:

1. Arduino UNO/ Raspberry Pi.
2. DHT11.
3. Connecting cable or USB cable.
4. Jumper wires.

CONNECTION DIAGRAM:



a) DHT11 sensor



b) DHT11 interfacing with Arduino

Step 1: Install the library for DHT in Arduino IDE.

- Open Arduino IDE and navigate to **Sketch > Include Library > Manage Libraries**.
- Search for "**DHTlib**" and install the "**DHTlib**" library in the Arduino IDE.

CODING:

Embedded C

```
#define dht_dpin 2
#include<dht.h> dht DHT;
void setup()
{
Serial.begin(9600);
Serial.println("Humidity and temperature\n\n");
}
void loop()
{
DHT.read11(dht_dpin);
Serial.print("Current humidity = ");
 Serial.print(DHT.humidity);
Serial.print("%
Serial.print("temperature = ");
Serial.print(DHT.temperature);
 Serial.println("C ");
delay(1000);
}
```

```
Output    Serial Monitor ×
Message (Enter to send message to 'Arduino Uno' on 'COM14')
Current humidity = 36.00%  temperature = 31.00C
Current humidity = 36.00%  temperature = 31.00C
Current humidity = 36.00%  temperature = 31.00C
Current humidity = 36.00%  temperature = 31.00C
Current humidity = 58.00%  temperature = 31.00C
Current humidity = 66.00%  temperature = 32.00C
Current humidity = 72.00%  temperature = 32.00C
Current humidity = 76.00%  temperature = 32.00C
Current humidity = 77.00%  temperature = 32.00C
}
```

You need to install PIP before you can install the Adafruit_DHT module: sudo pip3 install Adafruit_DHT

RESULT: Interfacing of DHT11 with Arduino UNO to print temperature and humidity readings has successfully executed.

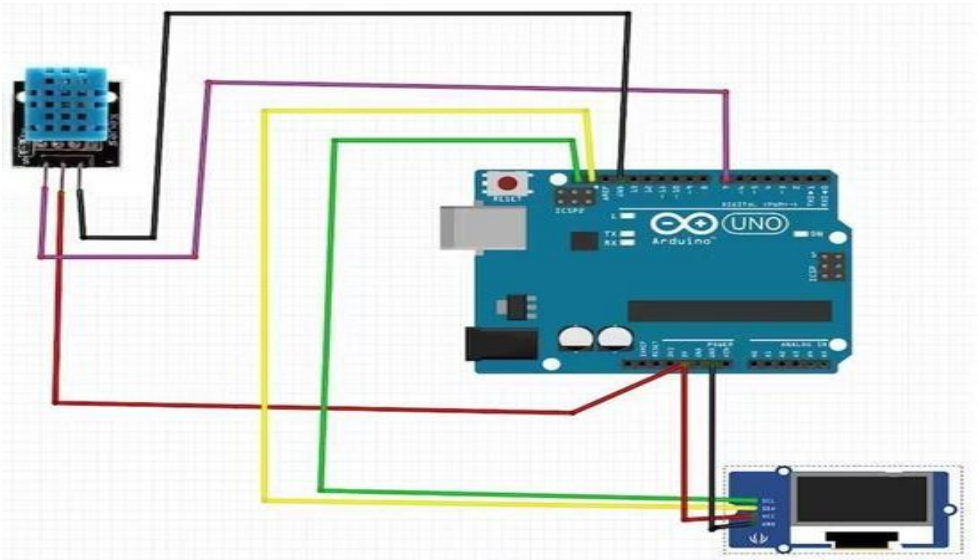
II.To interface OLED/I2C LCD with Arduino/Raspberry Pi and write a program to print temperature and humidity readings on it.

AIM: To write a program to interface OLED/ I2C LCD with Arduino to print temperature and humidity readings.

COMPONENTS REQUIRED:

1. Arduino UNO/ Raspberry Pi.
2. DHT11.
3. (0.96 Inch) I2C/IIC 4pin OLED Display Module / I2C LCD .
4. Connecting cable or USB cable.
5. Jumper wires.

CONNECTION DIAGRAM:



Steps to interface I2C OLED with Arduino:

Step 1: Install the library for OLED display in Arduino IDE.

- Open Arduino IDE and navigate to **Sketch > Include Library > Manage Libraries.**

- Search for "**SSD1306**" and install the "**SSD1306**" library from Adafruit in the ArduinoIDE.

Step 2: Import "Adafruit_GFX.h" & "Adafruit_SSD1306.h" header files in the code. Define header file in the code #include <Adafruit_GFX.h>,

#include <Adafruit_SSD1306.h>

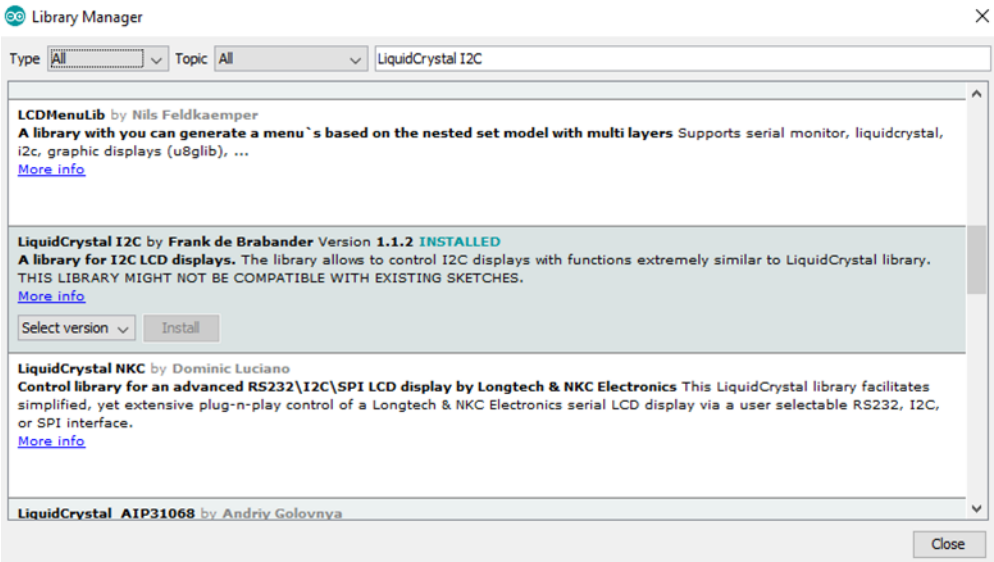
Step 3: Connect I2C OLED display device to Arduino as per the circuit diagram.

Initially run I2C scanner which will find the address of I2C module that is connected to Arduino. In this case address of I2C OLED module is **0x3C.**

Steps to interface I2C LCD with Arduino:

Step 1: Install the library for LCD display in Arduino IDE.

- Open Arduino IDE and navigate to **Sketch > Include Library > Manage Libraries.**
- Search for "**LiquidCrystal I2C**" and install the "**LiquidCrystal I2C**" library in the Arduino IDE.



Step 2: Import "LiquidCrystal_I2C.h" header file in the code.

- Define header file in the code #include <LiquidCrystal_I2C.h> .

Step 3: Connect I2C LCD display device to Arduino as per the circuit diagram.

- Initially run I2C scanner which will find the address of I2C module that is connected to Arduino. In this case address of I2C LCD module is **0x27.**

CODING:

Embedded C

Displaying Temperature and Humidity in the I2C LCD with Arduino

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C
 lcd(0x27,16,2);
#define dht_dpin 7
#include<dht.h>
dht DHT;
void setup()
{
```

```
Serial.begin(9600);
lcd.begin();
}
void loop()
{
DHT.read11(dht_dpin);
lcd.setCursor(0, 0);
lcd.print("Temp=");
lcd.print(DHT.temperature);
lcd.print("Celsius");
lcd.setCursor(0, 1);
lcd.print("Humidity=");
 lcd.print(DHT.humidity);
//print the humidity lcd.print("%");
delay(2000);
}
```

RESULT: Interfacing of OLED/ I2C LCD and DHT11 with Arduino UNO to print temperature and humidity readings has successfully executed.

EXPERIMENT – 3

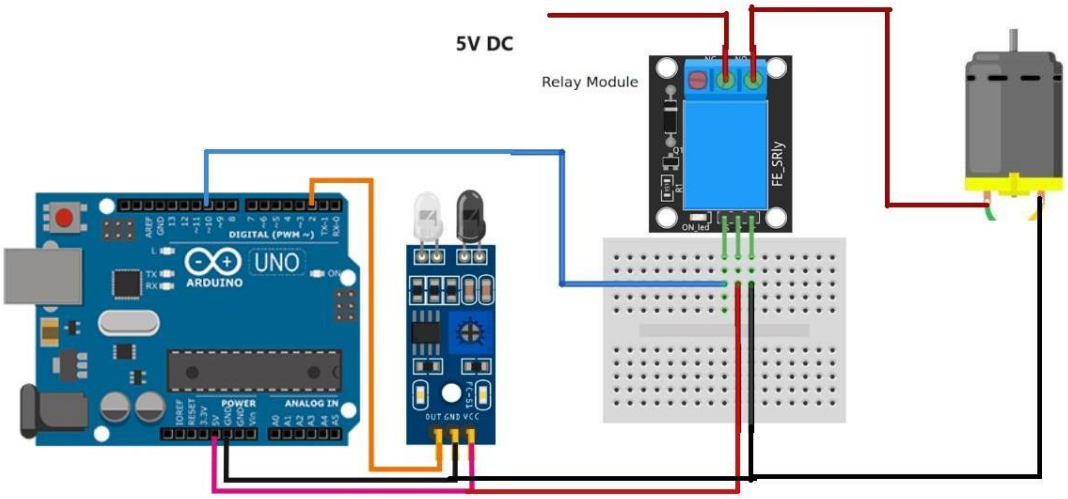
To interface motor using relay with Arduino/Raspberry Pi and write a program to turn ON motor when push button is pressed.

AIM: To write a program to turn ON motor when push button is pressed using relay with Arduino UNO.

COMPONENTS REQUIRED:

1. Arduino UNO.
2. Relay.
3. Resistor (10kΩ).
4. Motor.
5. Connecting cable or USB cable.
6. Breadboard.
7. Jumper wires.

CONNECTION DIAGRAM:



CODING:

Embedded C

```
void setup()
{
```

```
 pinMode(2,INPUT);
pinMode(10,OUTPUT); }
void loop()
{
 int sensor_state=digitalRead(2);
if(sensor_state==1)
{
   digitalWrite(10,HIGH);
}
else
{
   digitalWrite(10,LOW);
}
}
```

RESULT: Motor turn ON/OFF operation when push button is pressed using relay with Arduino UNO has successfully executed.

EXPERIMENT – 4

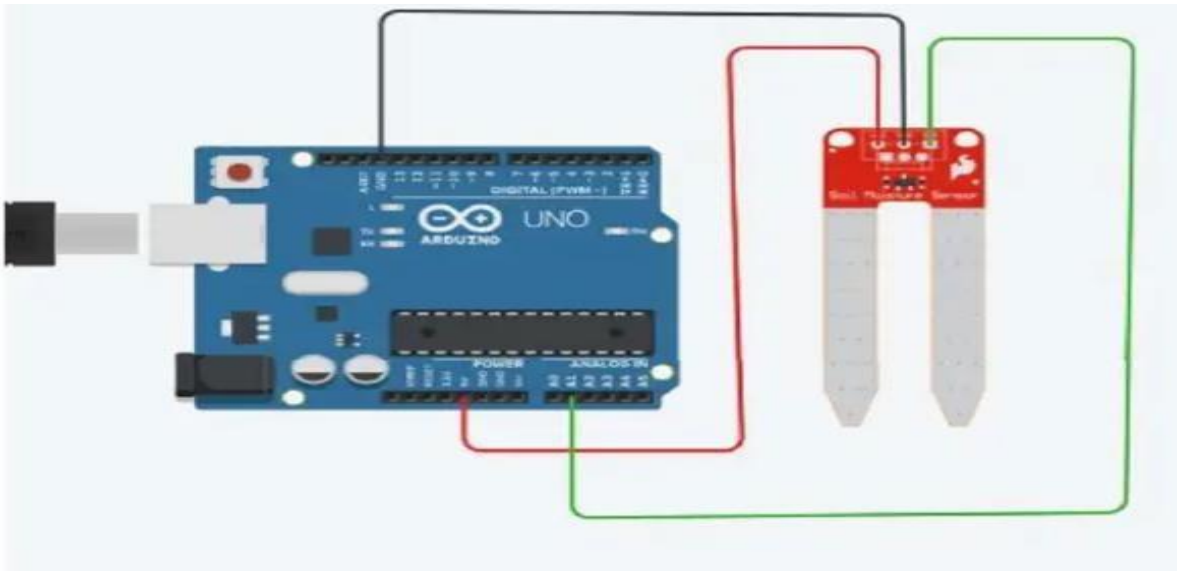
I. Write an Arduino/RaspberryPi program to interface the Soil Moisture Sensor.

AIM: Write a program to detect soil moisture using Arduino UNO by interfacing soil moisture sensor.

COMPONENTS REQUIRED:

1. Arduino UNO.
2. Soil Moisture sensor
3. Connecting cable or USB cable.
4. Breadboard.
5. Jumper wires.

CONNECTION DIAGRAM:



CODING:

Embedded C

```
int sensorpin = A0;
```

```
int sensorvalue = 0;
void setup()
{
Serial.begin(9600);
}
void loop()
{
sensorvalue = analogRead(sensorPin);
Serial.print("Soil Moisture Level:");
Serial.println(sensorvalue);

if (sensorValue<300)
{
   Serial.println("status : Soil is Wet");
}
 else if(sensorvalue<700)
{
   Serial.println("status : Soil is Moist");
}
else
{
   Serial.println("status : Soil is Dry");
}
delay(1000);
}
```

RESULT: Soil moisture detection by interfacing soil moisture sensor with Arduino UNO has successfully executed.

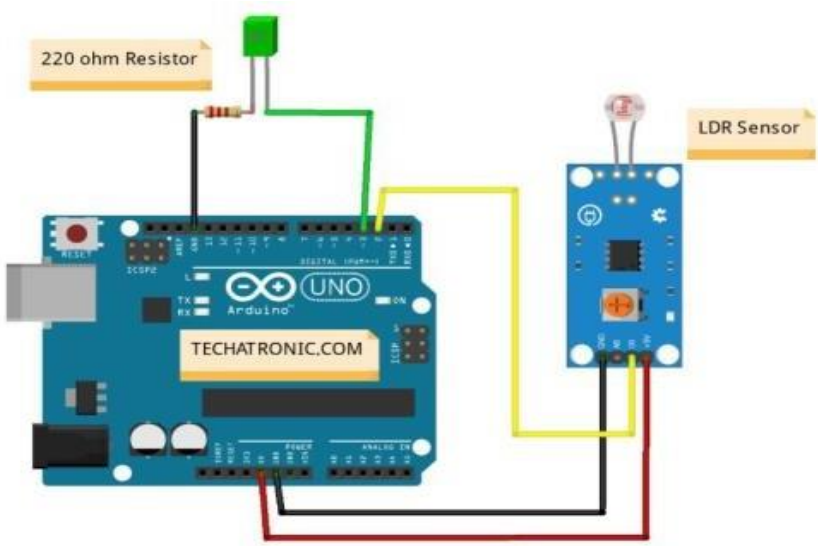
II. Write an Arduino/RaspberryPi program to interface the LDR/PhotoSensor.

AIM: Write a program to turn ON LED at sensor (LDR) detection using Arduino UNO.

COMPONENTS REQUIRED:

1. Arduino UNO.
2. LDR.
3. LED.
4. Resistor (100KΩ)
5. Connecting cable or USB cable.
6. Breadboard.
7. Jumper wires.

CONNECTION DIAGRAM:



CODING:

Embedded C

```
int ldrpin = 2;
 int ledpin=13;
```

```
void setup()
{
pinMode(ldrpin, INPUT);
pinMode(ledpin, OUTPUT);
Serial.begin(9600);
}
void loop()
{
int ldrValue = digitalRead(ldrPin);
Serial.print("ldrValue:");
Serial.println(ldrvalue);

if (ldrvalue == 1)
{
   digitalWrite(13, HIGH);
}else{
   digitalWrite(13, LOW);
   }
   delay(200);
}
```

RESULT: Controlling LED by interfacing LDR with Arduino UNO has successfully executed.

EXPERIMENT – 5

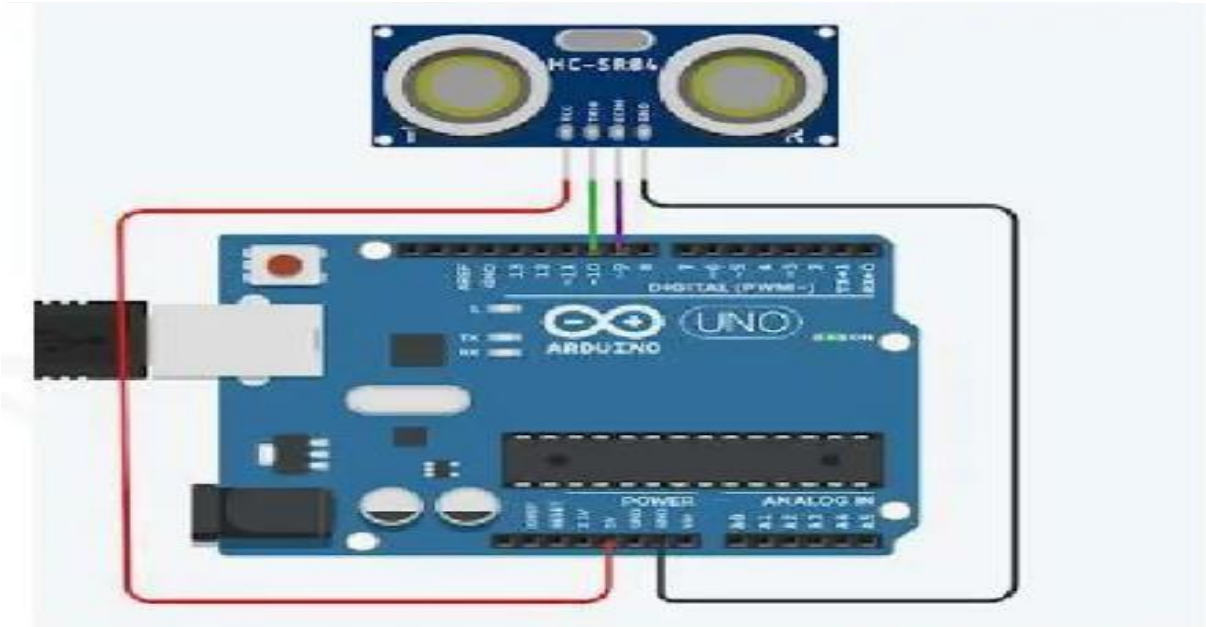
Write a program to interface an Ultrasonic Sensor with Arduino /Raspberry Pi.

AIM: To write a program to interface an ultrasonic sensor with arduino.

COMPONENTS REQUIRED:

6. Arduino UNO.
7. Ultrasonic sensor
8. Connecting cable or USB cable.
9. Breadboard.
10. Jumper wires.

CONNECTION DIAGRAM:



CODING:

Embedded C

```
const int trigPin = 9;
const int echoPin=10;

long duration;

float distance;

void setup()

 {

   Serial.begin(9600);
  pinMode(trigPin,OUTPUT);
  pinMode(echoPin,INPUT);
}


void loop() {
  digitalWrite(trigPin,LOW);
  delayMicroseconds(2);
   digitalWrite(trigPin,HIGH);
   delayMicroseconds(10);
   digitalWrite(trigPin, LOW);

  duration=pulseIn(echoPin,HIGH);
  distance = (duration*.0343)/2;
  Serial.print("Distance: ");
  Serial.println(distance);
  Serial.println("cm");
  delay(500);
}

}
```

RESULT: Soil moisture detection by interfacing soil moisture sensor with Arduino UNO has successfully executed.

EXPERIMENT – 6

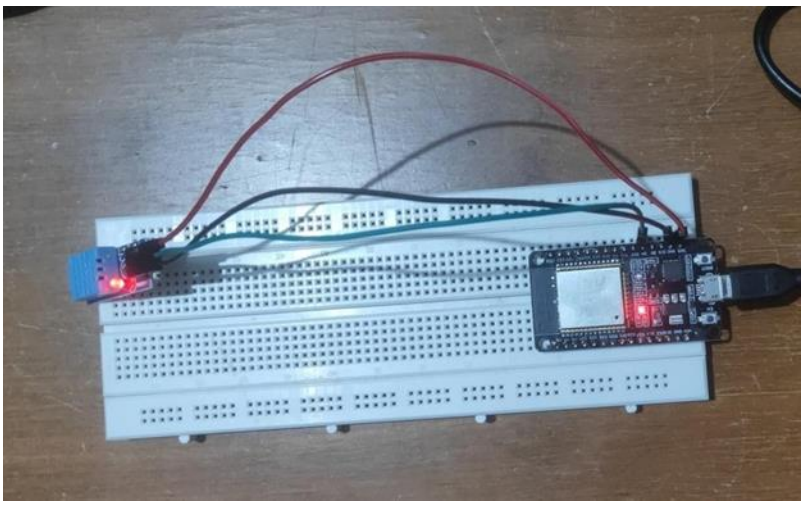
Write a program on Arduino/Raspberry Pi to upload temperature and humidity data to Thingspeak cloud.

AIM: To write a program on ESP32 to upload temperature and humidity value using DHT11 sensor to Thingspeak cloud

COMPONENTS REQUIRED:

1. ESP32.
2. DHT11.
3. Connecting cable or USB cable.
4. Breadboard.
5. Jumper wires.

CONNECTION DIAGRAM:

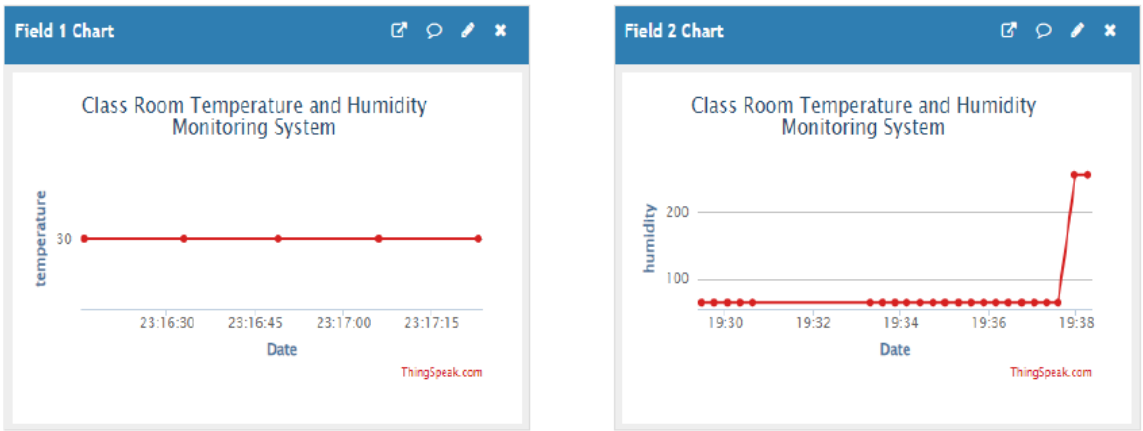


CODING:

```
//Interfacing DHT11 to ESP32 to upload temperature and humidity data tothingspeak cloud
#include <DHT.h>
#include<WiFi.h>
 #include<WiFiClient.h>
#include <ThingSpeak.h>
 #include <Adafruit_Sensor.h>
```

```
#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
const char* ssid = "GMIT";
const char* password = "DAVANGERE";
WiFiClient client;
unsigned long myChannelNumber = 12345;
const char * myWriteAPIKey = "ABCDEFGH";
uint8_t temperature, humidity;
void setup()
{
Serial.begin(9600);
 dht.begin();
delay(10);
// Connect to WiFi network
Serial.println();
Serial.println();
Serial.print("Connecting to ");
Serial.println(ssid);
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED)
{
delay(500);
Serial.print(".");
}
Serial.println("");
Serial.println("WiFi connected");
// Print the IP address
Serial.println(WiFi.localIP());
ThingSpeak.begin(client);
}
void loop()
{
temperature = dht.readTemperature();
humidity = dht.readHumidity();
Serial.print("Temperature Value is :");
Serial.print(temperature);
Serial.println("C");
Serial.print("Humidity Value is :");
Serial.print(humidity);
```

```
Serial.println("%");
// Write to Things peak. There are up to 8 fields in a channel, allowing you to store up to 8
different
ThingSpeak.writeField(myChannelNumber, 1, temperature, myWriteAPIKey);
ThingSpeak.writeField(myChannelNumber, 2, humidity, myWriteAPIKey);
delay(3000);
}
```



(b) Displaying temperature and humidity data on ThingSpeak cloud

RESULT: Sending temperature and humidity data using DHT11sensor to Thingspeak cloud with ESP32 has successfully executed.

EXPERIMENT – 7

Write a program on Arduino/Raspberry Pi to retrieve temperature and humidity data from Thingspeak cloud.

AIM: Write a program on ESP32 to retrieve temperature and humidity values using DHT11 sensor from Thingspeak cloud.

COMPONENTS REQUIRED:

1. ESP32.

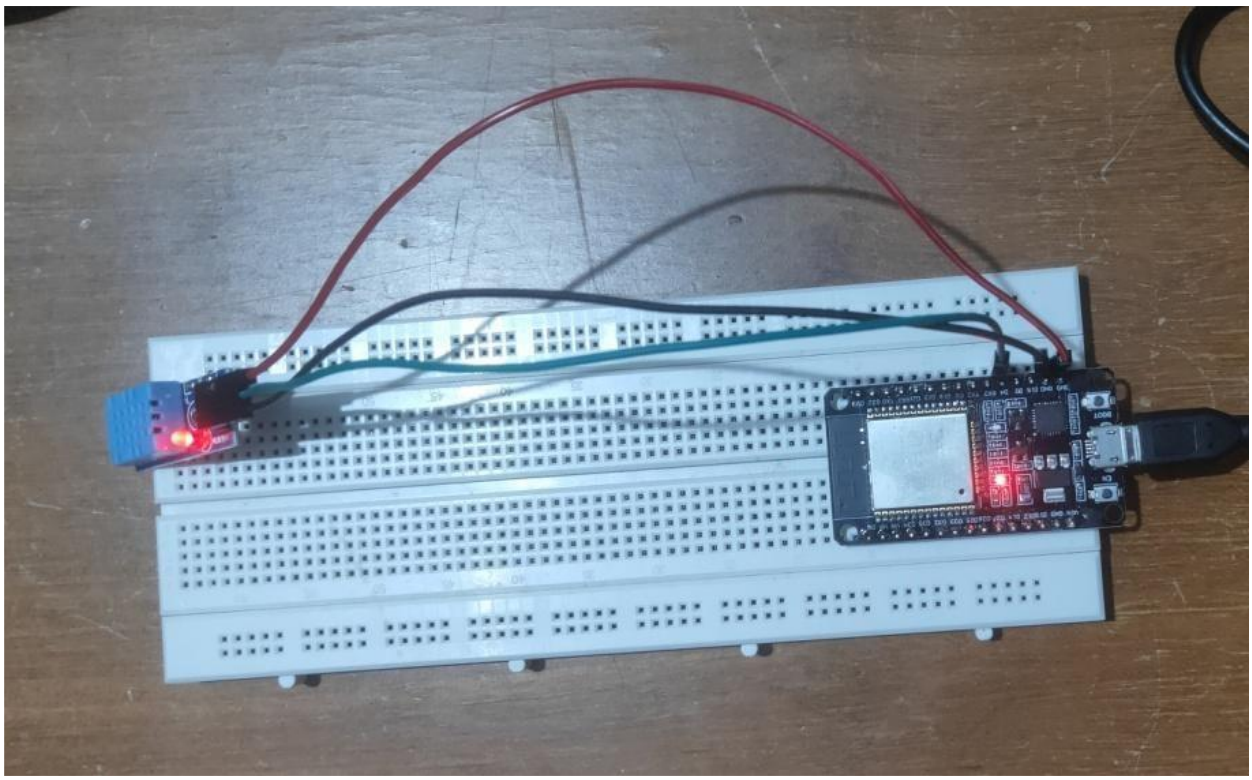
2. DHT11.

3. Connecting cable or USB cable.

4. Breadboard.

5. Jumper wires.

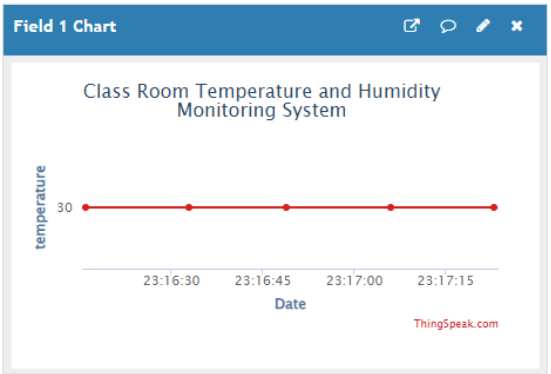
CONNECTION DIAGRAM:

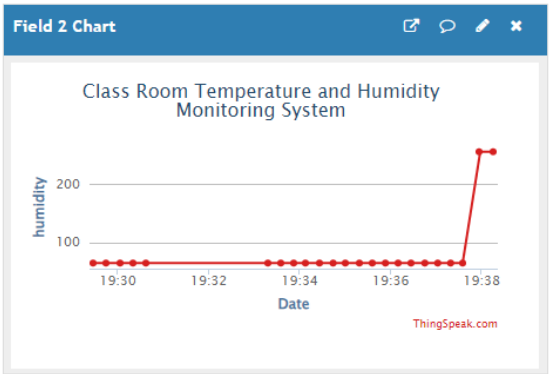


CODING:

Embedded C

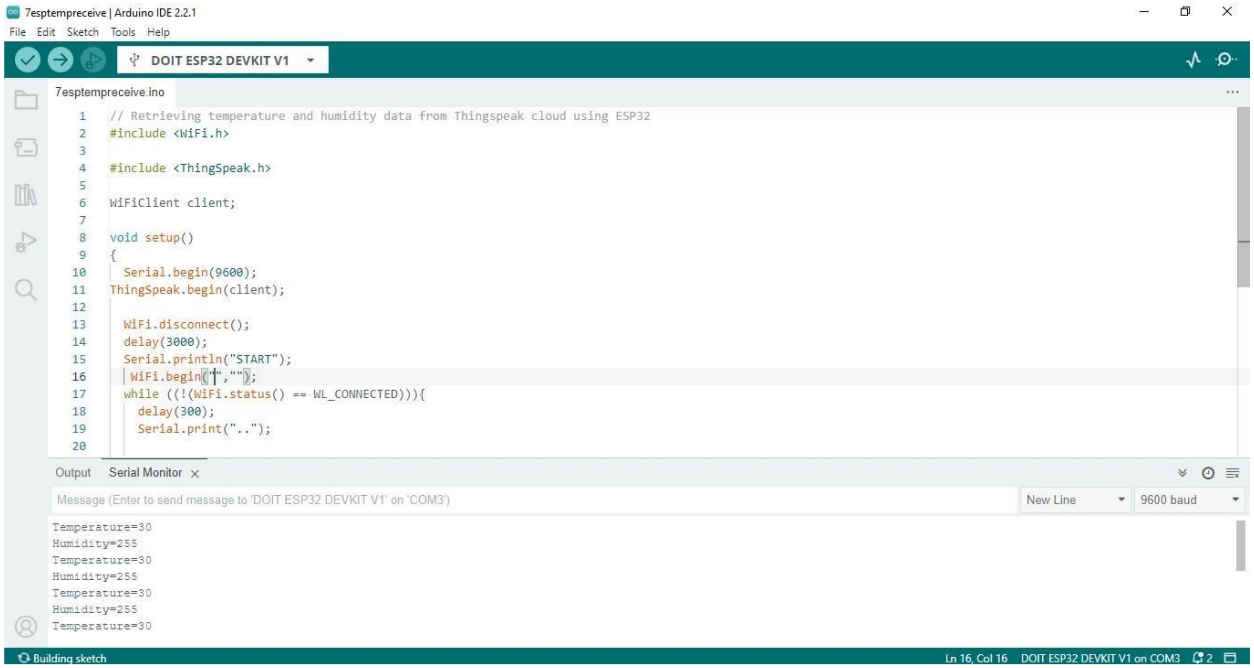
```
// Retrieving temperature and humidity data from Thingspeak cloud using ESP32 #include <WiFi.h>
#include <ThingSpeak.h>
WiFiClient client;
void setup()
{

WiFi.begin("GMIT","DAVANGERE");
while ((!(WiFi.status() == WL_CONNECTED)))
{
delay(300);
Serial.print("..");
}
Serial.println("Your IP is");
Serial.println((WiFi.localIP().toString()));
Serial.println("Connected");
}
voidloop()
{
Serial.print("Temperature=");
Serial.println((ThingSpeak.readIntField(12345,1,"ABCDEFGH")));
Serial.print("Humidity=");
Serial.println((ThingSpeak.readIntField(12345,2,"ABCDEFGH")));
delay(3000);
}
Serial.begin(9600);

ThingSpeak.begin(client);

WiFi.disconnect();

 delay(3000);
```





Available temperature and humidity data on ThingSpeak cloud

```
Serial.println("START");
```



Displaying temperature and humidity data on serial monitor of Arduino IDE from Thingspeak cloud

RESULT: Retrieving temperature and humidity data from Thingspeak cloud with ESP32 has successfully executed.

EXPERIMENT – 8

Write a program on Arduino/Raspberry Pi to publish temperature data to MQTT broker.

AIM: Write a program on ESP32 to publish temperature and humidity values using DHT11 sensor to MQTT broker.

COMPONENTS REQUIRED:

1. ESP32.
2. DHT11.
3. Connecting cable or USB cable.
4. Breadboard.
5. Jumper wires.

CONNECTION DIAGRAM:

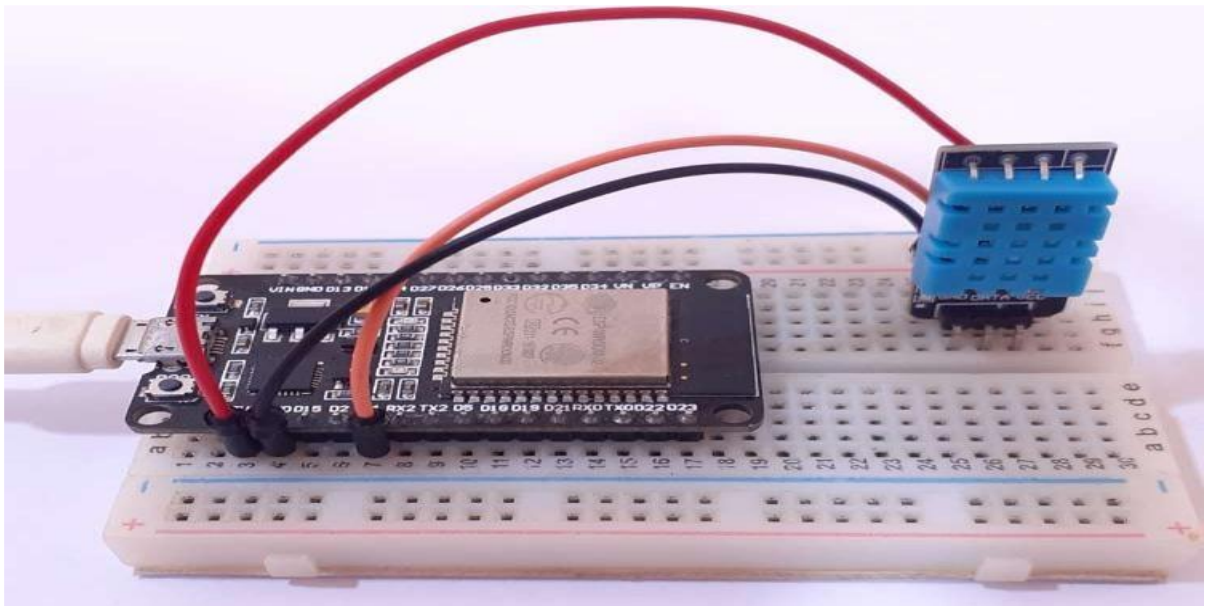


Fig 1 Interfacing DHT11 with ESP32

CODING:

Telegram Bot Configuration

1. Install Telegram on your laptop or phone.
2. Search for BotFather in Telegram and create a new bot.
3. Take note of the generated token, which will be used for interaction between the device and Telegram bot API.

Programming ESP8266

1. Download the Telegram Bot library for Arduino.
2. Open Arduino IDE and navigate to "Sketch" > "Include Library" > "Add .ZIP Li-brary" to add the downloaded library.
3. Copy the provided ESP8266 programming code into the Arduino IDE.
4. Replace <Your WiFi Name or SSID> with the name of your Wi-Fi network.
5. Replace <Your WiFi Password> with the password for your Wi-Fi network.
6. Replace <Token you get while creating the bot> with the token obtained from BotFather.

Embedded C

```
#include <ESP8266WiFi.h>
#include <WiFiClientSecure.h>
#include <TelegramBot.h>

#define LED 5


const char* ssid = "<Your WiFi Name or SSID>"; const
char* password = "<Your WiFi Password>";


const char BotToken[] = "<Token you get while creating the bot>";

WiFiClientSecure net_ssl;

TelegramBot bot (BotToken, net_ssl);

void setup()

{


  Serial.begin(115200);

  while (!Serial) {} //Start running when the serial is open
  delay(3000);
```

```
  Serial.print("Connecting WiFi.");
  Serial.println(ssid);

  while (WiFi.begin(ssid, password) != WL_CONNECTED)

  {

    Serial.print(".");
    delay(500);

  }

  Serial.println("");
  Serial.println("WiFi connected");
  bot.begin();

  pinMode(LED, OUTPUT);

}

void loop()

{

  message m = bot.getUpdates(); // Read new messages if
  (m.text.equals("on"))

  {

    digitalWrite(LED, 1);
    bot.sendMessage(m.chat_id, "LED is ON");
    Serial.println("LED is ON");

  }

  else if (m.text.equals("off"))

  {

    digitalWrite(LED, 0);
    bot.sendMessage(m.chat_id, "LED is OFF");
    Serial.println("LED is OFF");

  }

}
```

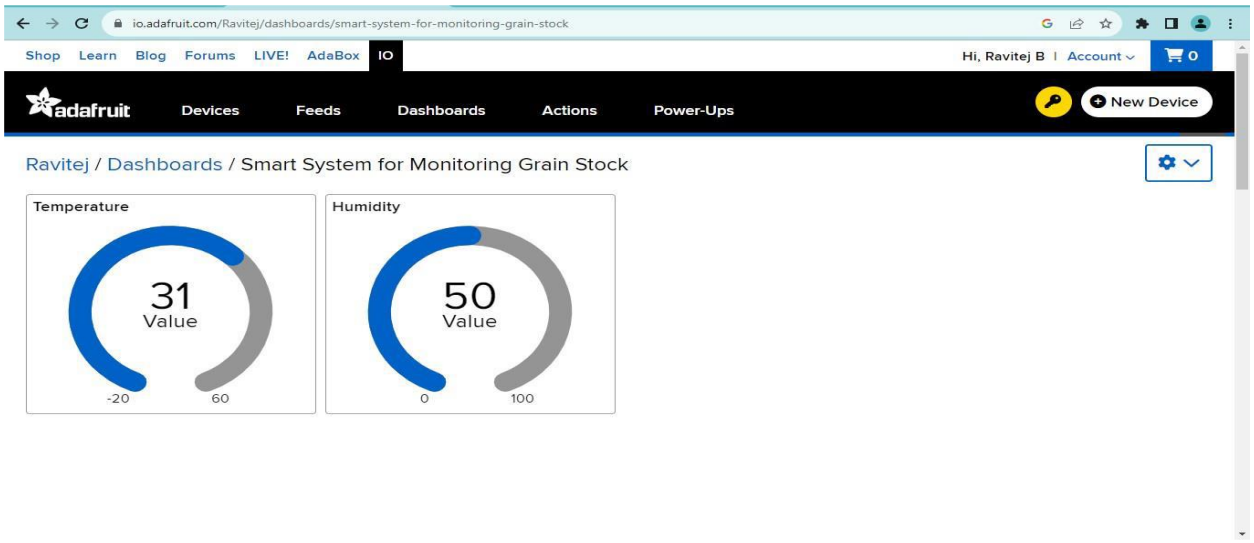


Fig: Displaying temperature and humidity data on AdafruitIO platform

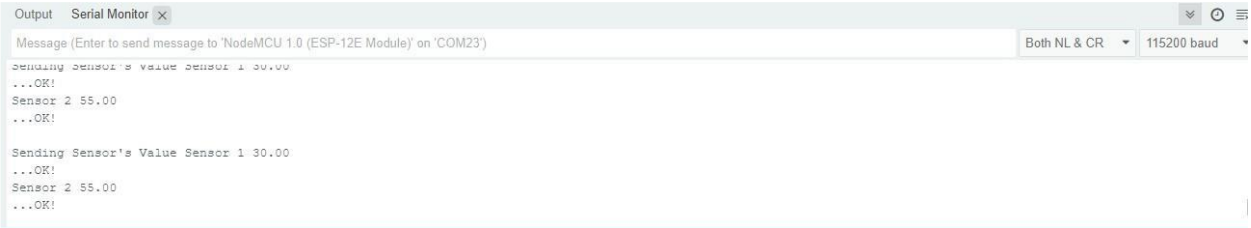


Fig: Displaying temperature and humidity data on serial monitor of ArduinoIDE

RESULT: Published temperature and humidity data to MQTT broker with ESP32 has successfully executed.

EXPERIMENT – 9 & 11

Write a program on NodeMCU to publish temperature data to MQTT broker.

Write a program to create a TCP server on Arduino /Raspberry Pi and respond with humidity data to the TCP client when requested

AIM: To write a program on Node MCU to publish/subscribe temperature data to MQTT broker.

The Messaging Queuing Telemetry Transport (MQTT) protocol is a publilsh/subscribe messaging protocol that works on top of your home TCP/IP network. It provides a fast and reliable way to connect your IOT devices together and create responses to messages.

- Every MQTT network needs a broker.
- The broker is the heart of the MQTT network and is just a server that receives information from devices, also known as clients, and distributes that information to other clients based on the topic.

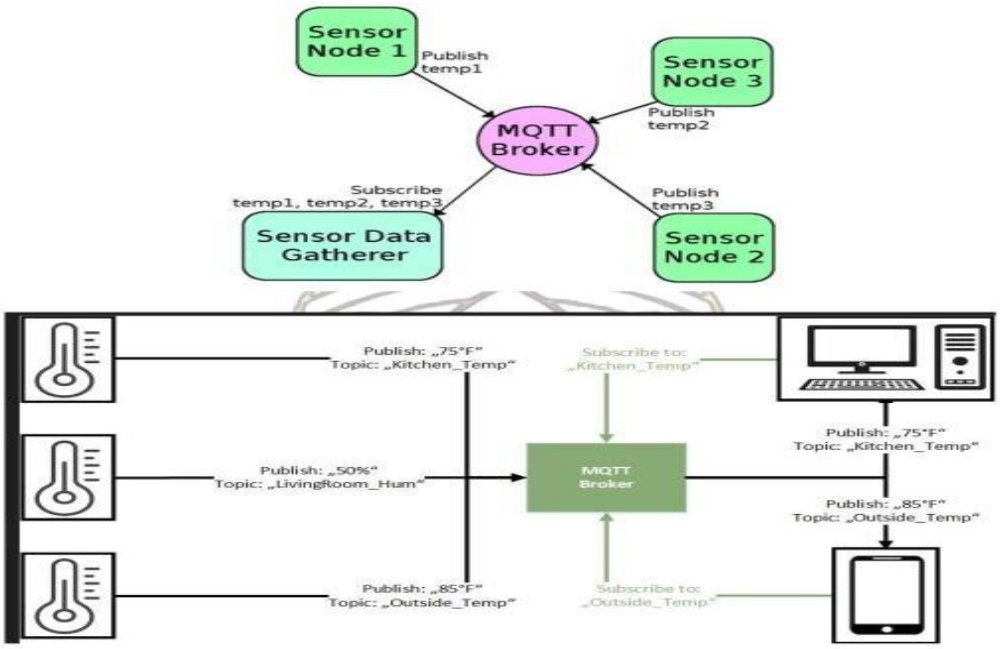


Fig 1 Interfacing DHT11 with ESP32**CODING:**

Embedded C

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
//=======include DHT SENSOR===========
#include      <dht11.h>
#define  DHT11PIN  0
dht11 DHT11;
//=========================

// Update these with values suitable for your network. const char*
ssid = "abc";
const char* password = "11111111";
const char* mqtt_server = "test.mosquitto.org"; WiFiClient
espClient;
PubSubClient client(espClient);
unsigned long lastMsg = 0; #define
MSG_BUFFER_SIZE (50) char
msg[MSG_BUFFER_SIZE]; int value
= 0;
void setup_wifi()
{

delay(10); Serial.println();
Serial.print("Connecting to ");
Serial.println(ssid);
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED)
{

delay(500); Serial.print(".");
}

randomSeed(micros()); Serial.println("");
Serial.println("WiFi connected");
Serial.println("IP address: ");
Serial.println(WiFi.localIP());
}

void callback(char* topic, byte* payload, unsigned int length)
```

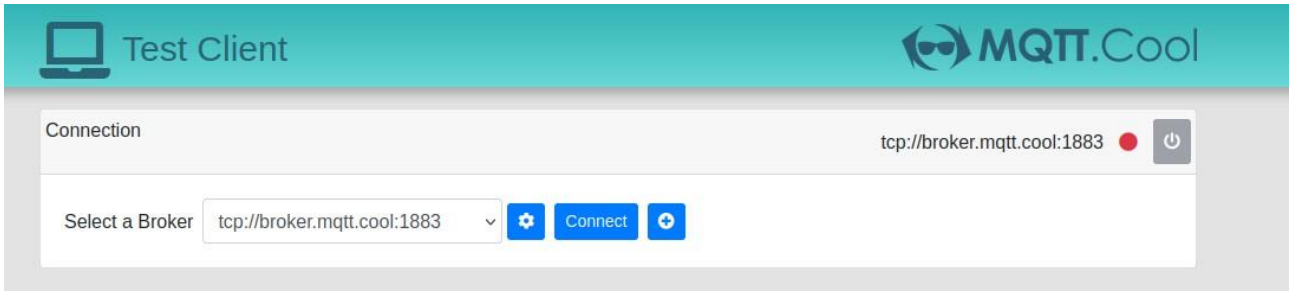
```
{

Serial.print("Message arrived [");
Serial.print(topic);
Serial.print("] ");
for (int i = 0; i < length; i++)
{

Serial.print((char)payload[i]);
}

Serial.println();
// Switch on the LED if an 1 was received as first character if
((char)payload[0] == '1')
{

digitalWrite(BUILTIN_LED, LOW);
}

else
{

digitalWrite(BUILTIN_LED, HIGH);
}

}

void reconnect()
{

// Loop until we're reconnected
while (!client.connected())
{

Serial.print("Attempting MQTT connection...");
// Create a random client ID
String clientId = "ESP8266Client-"; clientId +=
String(random(0xffff), HEX);
// Attempt to connect
if (client.connect(clientId.c_str()))
{Serial.println("connected");
```

```
// Once connected, publish an announcement... client.publish("device/temp",
"MQTT Server is Connected");
// ... and resubscribe
client.subscribe("device/led");
}

else
{

Serial.print("failed, rc=");
Serial.print(client.state()); Serial.println(" try
again in 5 seconds"); delay(5000);
}

}

}

void setup()
{

pinMode(BUILTIN_LED, OUTPUT);
Serial.begin(115200); setup_wifi();
client.setServer(mqtt_server, 1883); client.setCallback(callback);
// Initialize the BUILTIN_LED pin as an output
}

void loop()
{

int chk = DHT11.read(DHT11PIN); if
(!client.connected())
{

reconnect();
}

client.loop();
unsigned long now = millis(); if
(now - lastMsg > 2000)
{
```

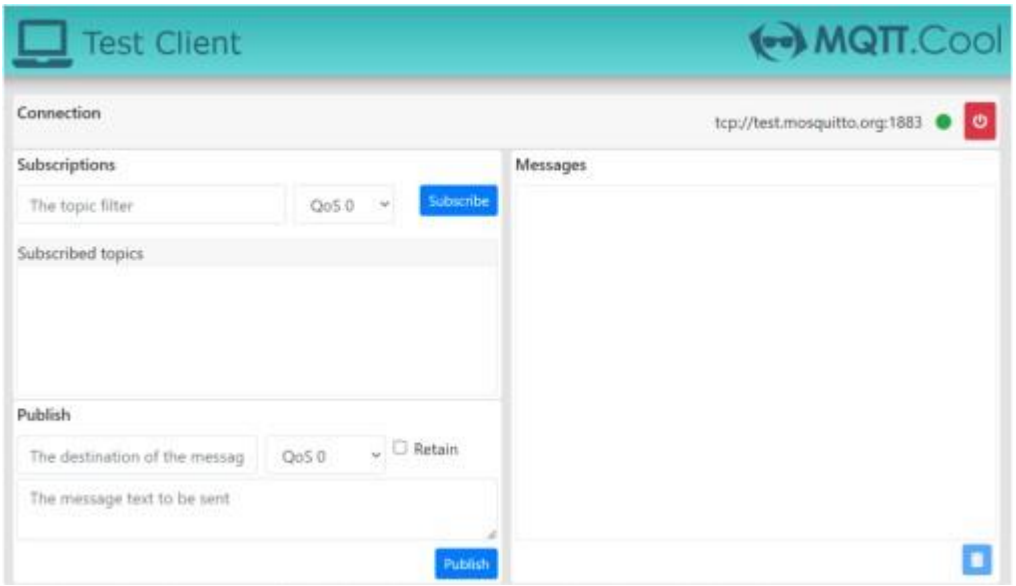
```
lastMsg = now;
float t = DHT11.temperature;
snprintf (msg, MSG_BUFFER_SIZE, "Temperature is :%f", t); Serial.print("Publish message:
");
Serial.println(msg);
client.publish("device/temp", msg);
}

}
```

Note:

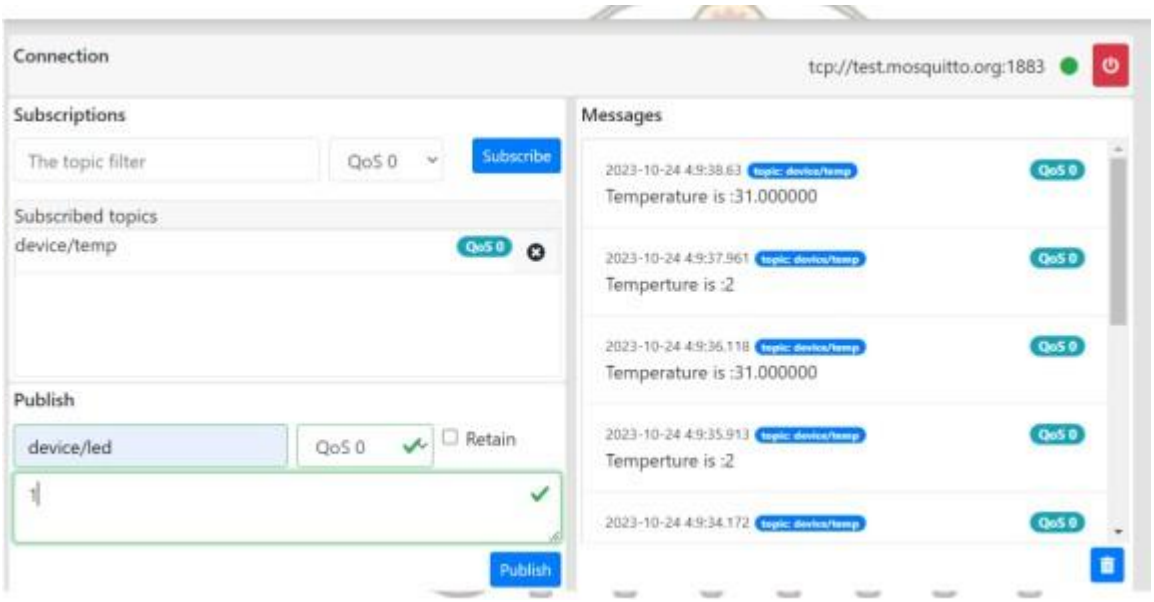
1. Browse the link :https://testclient-cloud.mqtt.cool/
2. Select tcp://test.mosquitto.org:1883 & click on connect



3. Connection window



4. Type device/temp and subscribe, then you will get the temperature reading in the message window
5. To control the LED of Node MCU, type device/led and click on publish
6. To turn on LED, type 1 and click on publish
7. To turn off LED, type 0 and click on publish



8. To know more browse and watch : https://www.youtube.com/watch?v=c1st5cVRRzo

EXPERIMENT – 10

Write a program to create UDP server on NodeMCU and respond with humidity data to UDP client when requested.

AIM: To write a program to create TCP server on Node MCU and respond with humidity data to TCP client when requested.

Embedded C

```
/*Create a TCP Server on ESP8266 NodeMCU. TCP Socket Server Send Receive
*/

#include <ESP8266WiFi.h>
//#define SendKey 0 //Button to send data Flash BTN on NodeMCU int port =
8888; //Port number
//=======include DHT SENSOR===========
#include      <dht11.h>
#define  DHT11PIN  0
dht11 DHT11;
//=======================

WiFiServer server(port);
//Server connect to WiFi Network
const char *ssid = "abc"; //Enter your wifi SSID
const char *password = "11111111"; //Enter your wifi Password int count=0;
void setup()
{

Serial.begin(115200);
Serial.println();
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password); //Connect to wifi
// Wait for connection
Serial.println("Connecting to Wifi");  while
(WiFi.status() != WL_CONNECTED)
{

delay(500); Serial.print(".");
delay(500);
```

```
  pinMode(LED, OUTPUT);
}
void loop()
{
}
Serial.println("");
Serial.print("Connected to ");
Serial.println(ssid); Serial.print("IP
address: ");
Serial.println(WiFi.localIP());
server.begin();
Serial.print("Open Telnet and connect to IP:");
Serial.print(WiFi.localIP());
Serial.print(" on port "); Serial.println(port);
}
void loop()
{
int chk = DHT11.read(DHT11PIN); int
key;
WiFiClient client = server.available(); if
(client)
{
if(client.connected())
{
Serial.println("Client Connected");
}
while(client.connected())
{
while(client.available()>0)
{
// read data from the connected client
```

```
Serial.write(client.read()); Serial.print("Humidity
(%): "); Serial.println((float)DHT11.humidity, 2);
}

//Send Data to connected client while(Serial.available()>0)
{

client.write(Serial.read());
}

}


client.stop();
Serial.println("Client disconnected");
}

}

//==============================================
```

Note:

1. Upload the program & find the IP Address in the serial com window
2. Open the packet sender software give the “IP Address” & Port address “8888” and select TCP
3. To get the humidity data, type 1 in the ASCII box & click on send, then observe the output in the serial window

EXPERIMENT – 12

Write a program on NodeMCU to subscribe to MQTT broker for temperature data and print it

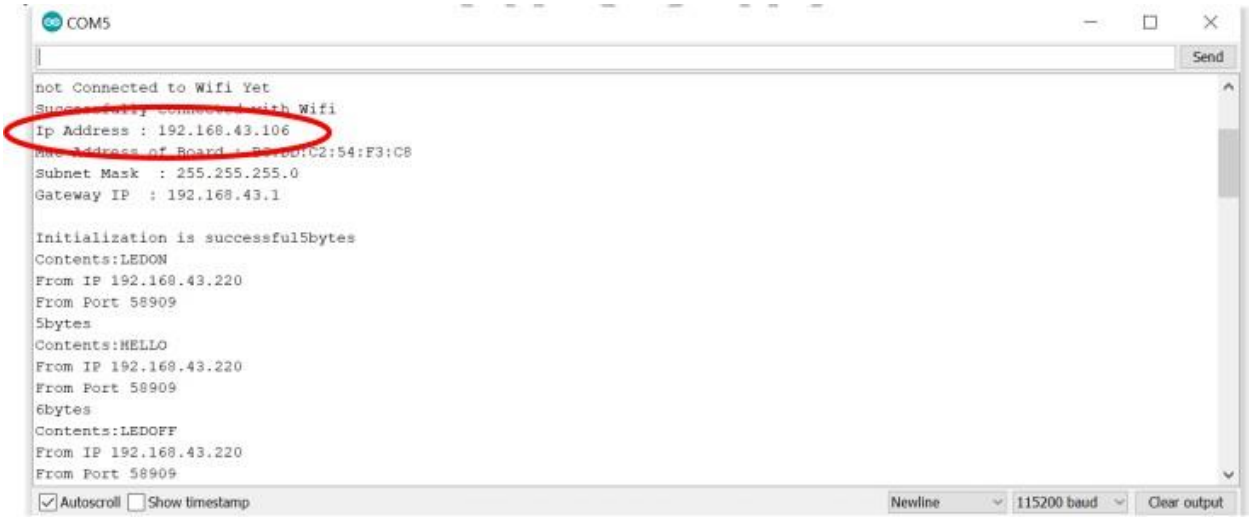
AIM: To write a program to create UDP server on NodeMCU and respond with humidity data to UDP client when requested.

Embedded C

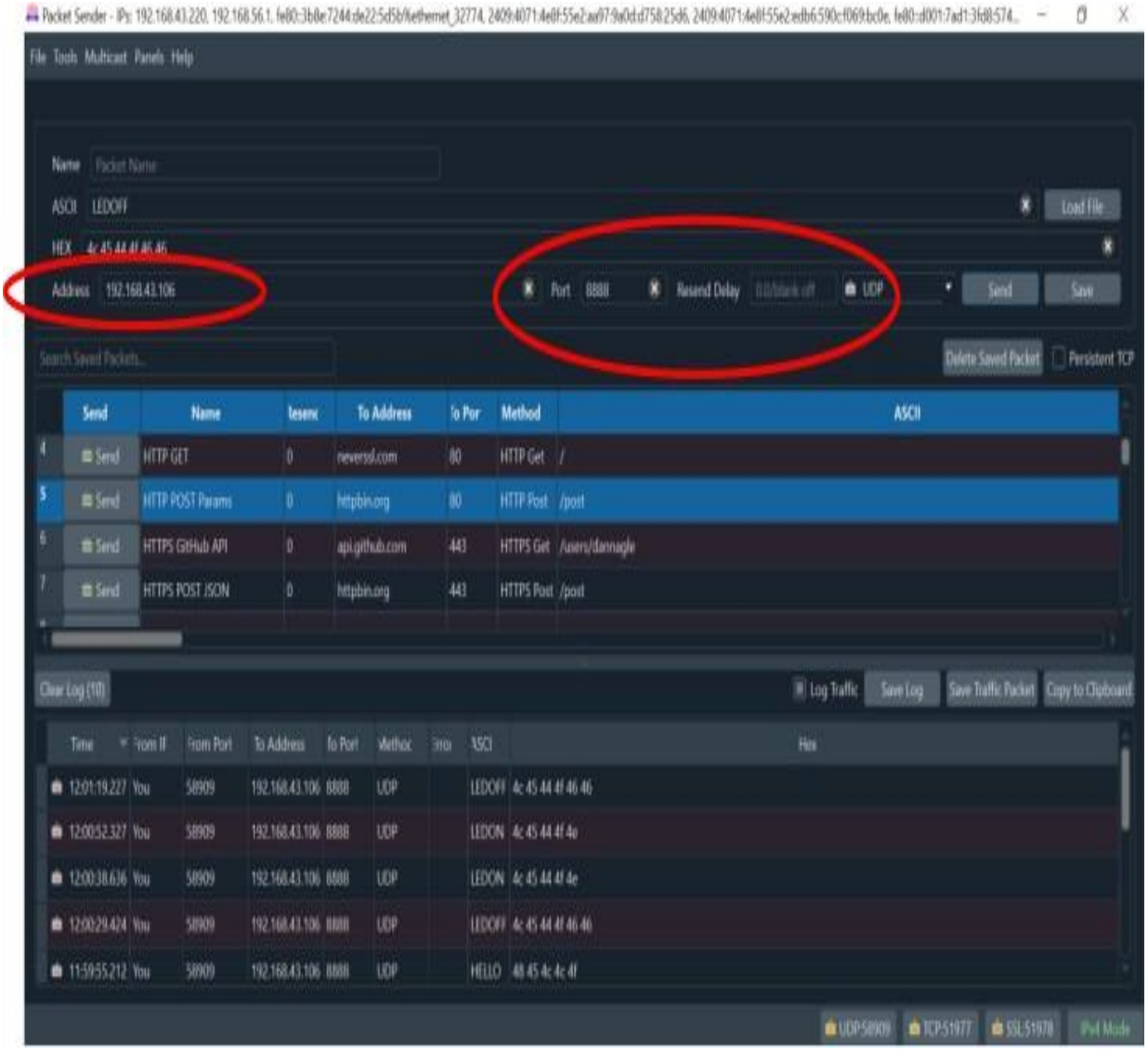
```
#include <ESP8266WiFi.h>
#include <WiFiUdp.h>
#define SOCKET 8888 // define udp port #define
BUFFERLEN 255 // length of buffer
//=======include DHT SENSOR===========
#include      <dht11.h>
#define  DHT11PIN  0
dht11 DHT11;
//=======================

char inBUFFER[BUFFERLEN]; // for storing receiving packet char
outBUFFER[BUFFERLEN]; // for storing sending packets WiFiUDP UDP;
const char ssid[]= "abc";
const char password[]="11111111"; void
setup() {   Serial.begin(115200);
Serial.print("Trying to connect to Wifi");
Serial.print(ssid); WiFi.begin(ssid,password);
while(WiFi.status() != WL_CONNECTED)
{

delay(500); Serial.print("\n");
Serial.print("not Connected to Wifi Yet");
}

Serial.print("\n");
Serial.println("Successfully connected with Wifi"); Serial.print("Ip
Address : "); Serial.println(WiFi.localIP());
}

Serial.print("Mac Address of Board : ");
Serial.println(WiFi.macAddress()); Serial.print("Subnet Mask
```

```
: "); Serial.println(WiFi.subnetMask()); Serial.print("Gateway
IP : "); Serial.println(WiFi.gatewayIP());
if ( UDP.begin(SOCKET))
{

Serial.print("\n"); Serial.print("Initialization is
successful");
}

else
{

Serial.print("Initialization is Failed");
}

pinMode(LED_BUILTIN, OUTPUT);
}

void loop()
{

int chk = DHT11.read(DHT11PIN);
int packetsize = 0; // tp store size of packet
packetsize = UDP.parsePacket(); // to check if there is any UDP packet if (packetsize)
{

UDP.read(inBUFFER,BUFFERLEN); // read udp packet and store in inbuffer
inBUFFER[packetsize] = '\0' ; // read untill null
if (strcmp("DATA",inBUFFER) == 0)
{

digitalWrite(LED_BUILTIN, LOW);
Serial.print("Humidity (%): ");
Serial.println((float)DHT11.humidity, 2);
}

if (strcmp("NODATA",inBUFFER) == 0)
{

digitalWrite(LED_BUILTIN, HIGH);// LED TURNS OFF
}
```

```
Serial.print(packetsize); Serial.println("bytes");
}

Serial.print("Contents:");
Serial.println(inBUFFER); Serial.print("From
IP "); Serial.println(UDP.remoteIP());
Serial.print("From Port ");
Serial.println(UDP.remotePort());
}

}
```



1. Upload the program & find the IP Address in the serial com window
2. Open the packet sender software give the “IP Adress” & Port address “8888” and select UDP
3. To get the humidity data, type “DATA” in the ASCII box & click on send, then observe the output in the serial window
4. To turnoff the LED of Node MCU, type “NODATA” in the ASCII box & click on send, then observe the LED status


File Tools Multicast Panels Help
Name
ASCII LEDOFF
HEX 4c 45 44 4f 46 46
Address 192.168.43.106
Port 8888
Resend Delay
UDP
Send
Save
Load File
Delete Saved Packet
Persistent TCP
Send Name To Address Method ASCII
HTTP GET neversl.com 80 HTTP Get /
HTTP POST Params 0 httpbin.org 80 HTTP Post /post
HTTPS GitHub API 0 api.github.com 443 HTTPS Get /users/dannagle
HTTPS POST JSON 0 httpbin.org 443 HTTPS Post /post
Clear Log (10)
Log Traffic
Save Log
Save Traffic Packet
Copy to Clipboard
Time From IF From Port To Address To Port Method ASCII Hex
12:01:19.227 You 58909 192.168.43.106 8888 UDP LEDOFF 4c 45 44 4f 46 46
12:00:52.327 You 58909 192.168.43.106 8888 UDP LEDON 4c 45 44 4f 4e
12:00:38.636 You 58909 192.168.43.106 8888 UDP LEDON 4c 45 44 4f 4e
12:00:29.424 You 58909 192.168.43.106 8888 UDP LEDOFF 4c 45 44 4f 46 46
11:59:55.212 You 58909 192.168.43.106 8888 UDP HELLO 48 45 4c 4c 4f
UDP:58909
TCP:51977
SSL:51978
IPv4 Mode